An operational decision framework for dual-source mixed-flow PECVD rotating-chamber cluster tools

Lizhen Shao[a,*], Lincong Wei[a], Wei Wang[a]

[a]*Department of Control Science and Engineering, University of Science and Technology Beijing, No. 30 Xueyuan Road, Haidian District, Beijing 100083, China*

Abstract

This study develops an operational decision framework for PECVD rotating-chamber cluster tools supplied by product wafers from the atmospheric side and reusable dummy wafers from vacuum-side storage. The two material flows share robots and multi-position chambers, coupling material availability, full occupancy, processing, and periodic cleaning. A two-stage mixed-integer linear programming model represents two processing modes and the complete routes of both wafer types. Structure-based initial schedules and learning-enhanced cut selection support time-limited solution search. Computational experiments show that, under prescribed PW layouts, the preferred mode depends on lot size, cleaning interval, and complete-chain segmentation. For a 16-wafer configuration without cleaning, chained processing reduces makespan by 4.87% relative to parallel processing. The two-stage workflow improves waiting and processing cadence, while precomputed structured initial schedules increase feasible-schedule availability during the subsequent timed solve. The framework provides a consistent basis for mode selection, dummy-wafer allocation, and event scheduling in dual-source mixed-flow equipment.

Keywords: semiconductor manufacturing system, cluster tool, dual-source material flow, operational decision, mixed-integer linear programming, learning-enhanced exact optimization

1. Introduction

Three-dimensional device structures and multilayer thin-film integration impose increasing demands on deposition uniformity, processing capability, and equipment output per unit floor area [1]. Plasma-enhanced chemical vapor deposition (PECVD) is an important thin-film deposition process. Cluster tools (CTs) integrate process modules, vacuum-handling robots, and load locks so that wafers can be loaded, processed, and returned within a controlled environment. Tool output depends on coordination between processing and handling; load-lock switching, chamber cleaning, and residency limits further constrain feasible operation [2].

Multi-space process modules and multi-finger robots provide a structural basis for parallel processing within an equipment cell [3, 4]. This study considers a four-position rotating-chamber CT for PECVD. Rotation changes the positions accessible to the robot and connects multi-wafer processing with paired handling. In this configuration, PECVD specifies the process and recipe durations, while chamber rotation enables position changes and transitions between consecutive processing cycles. Their combination determines processing organization and loading and unloading order.

The tool uses product wafers and reusable dummy wafers. Product wafers initially occupy atmospheric-side load ports and enter the vacuum domain through the atmospheric transfer robot (ATR), aligner (AL), and load locks (LLs). Dummy wafers are preloaded in a storage unit mounted in the vacuum domain and are accessed directly by the vacuum transfer robot (VTR). We call this supply structure dual-source. Under the full-occupancy recipe considered here, dummies fill positions not occupied by products and provide the occupancy required for cleaning. Their inventory and reuse times therefore constrain subsequent product processing.

The two wafer types have different origins and circulation routes but share the VTR and rotating-chamber positions, forming mixed-flow operation (Fig. 1). Product flow connects atmospheric-side batch input to processed-wafer return; dummy flow circulates between vacuum storage and the chambers. Full occupancy, dummy inventory, and shared-resource timing couple these routes. Admission of a product wafer pair thus depends jointly on product arrival, dummy availability, chamber state, and robot position. Scheduling must represent the supply, convergence, and return of both material flows.

Two preset modes are considered: 4×1 parallel processing and 2×2 chained processing. The former processes four wafers synchronously in one exposure. The latter uses wafer pairs (PWs) as handling and occupancy units, alternating loading and unloading between adjacent cycles so that each product wafer participates in two consecutive exposures. Conditional on both recipes meeting the relevant product-process requirements, the comparison holds cumulative nominal processing time per wafer constant. Differences in cycle count, occupancy, and exchange order produce different cleaning positions and dummy

*Corresponding author.

Email addresses: lshao@ustb.edu.cn (Lizhen Shao), m202420836@xs.ustb.edu.cn (Lincong Wei)

demands for the same lot. Cleaning occupies chambers and changes return times, thereby affecting subsequent chain transitions. The resulting decision problem coordinates modes, dual-source material flow, and equipment actions under residency and capacity constraints to shorten finite-batch makespan and improve waiting and processing cadence.

Manufacturing-system research coordinates processing resources, logistics, and production data through multi-agent architectures, joint layout and scheduling, and transferable control representations [5–7]. In semiconductor manufacturing, industrial data spaces and historical decision knowledge support data organization and dynamic dispatching [8, 9], while quality constraints guide route selection, equipment allocation, and job sequencing [10, 11]. This study addresses how production decisions are realized within an equipment cell: batch and recipe inputs must become action sequences satisfying material-supply, chamber-occupancy, and handling constraints.

CT research established analytical foundations for throughput and residency feasibility. Analytical models relate processing and handling times to throughput [12]; linear programming, heuristics, and timed Petri nets produce feasible or optimal schedules under residency constraints [13–15]. Subsequent models address revisiting processes, multiple wafer types, and selectable process modules [16–18]. Steady-state analysis of four-finger robots and four-space modules, together with joint optimization of two-type wafer releases and robot actions, provides a basis for more complex tool structures [4, 19]. Finite batches additionally require chain initialization, termination, and partial-batch filling, making actual dummy-supply and return times constraints on subsequent processing.

Cleaning and shared resources further couple material routes. Task-sequence enumeration, partial loading and timing control, virtual-wafer occupancy, and chamber-state-dependent triggers coordinate cleaning with production [20–23]. Shared-buffer synchronization and dynamic action screening address deadlock during failure shutdown and noncyclic operation [24, 25]. Waiting constraints also affect equipment output and upstream control [26]. In the present configuration, cleaning changes dummy occupancy, chain segmentation, and shared-VTR service order simultaneously, requiring joint scheduling of complete product routes and closed-loop dummy routes.

Learning can select CT actions directly or support mathematical optimization. Deep reinforcement learning with look-ahead search addresses noncyclic control, while DDPG has been used to adjust PVD processing times in simulation [27, 28]. Here, recipe durations are inputs and learning is embedded in MILP cut selection. Relevant approaches include reinforcement-learning selection, cut ranking, and look-ahead imitation [29–31]; HEM jointly selects cut counts and ordered sequences [32]. This hierarchical policy provides a basis for incorporating equipment-variable roles while retaining the constraint model.

This study examines the joint effects of product supply, dummy reuse, and cleaning organization. Under a prescribed mode and PW layout, dummy return times constrain subsequent full-occupancy processing, while complete chains determine the available cleaning boundaries. The model jointly converts these material dependencies and shared-resource orders into event times. The contributions are:

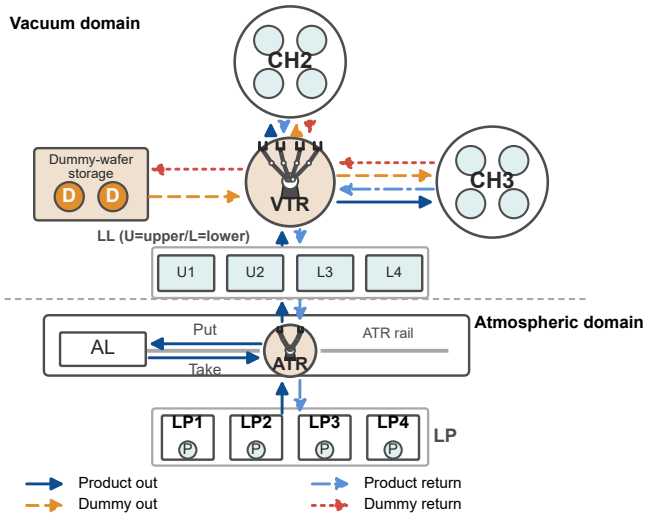


Figure 1: Dual-source mixed-flow rotating-chamber CT. Products enter the vacuum domain from atmospheric-side LPs through ATR, AL, and LLs; dummies enter from vacuum storage through VTR. Solid, dash-dot, dashed, and dotted arrows indicate product outbound, product return, dummy outbound, and dummy return routes, respectively.

1. A material-flow model couples atmospheric-side product supply and vacuum-side dummy reuse through full chamber occupancy, cleaning, and shared handling resources. It generates complete finite-batch event schedules for both preset processing modes.

2. A two-stage MILP first minimizes product-return makespan, then refines waiting and cadence while retaining the makespan bound and principal discrete order. Mode comparisons quantify how lot size and cleaning interval change preferred operation through complete-chain segmentation under prescribed PW layouts.

3. A structure-aware solution framework combines SPBS initialization with a role-aware hierarchical cut selector. Matched-budget experiments separately evaluate initialization, timing refinement, and cut selection, identifying their contributions to feasible-schedule availability and solution quality.

Section 2 presents the equipment architecture, material flows, and two-stage model. Section 3 develops SPBS and structure-aware cut selection. Section 4 evaluates mode choice, schedule quality, feasible-schedule delivery, solver status, and changes in scale and parameters. Section 5 concludes.

2. Manufacturing-system architecture, material flow, and two-stage operational model

2.1. Equipment context, configuration, and event outputs

Equipment-cell scheduling connects production configuration to resource actions. Within the CIM perspective, material and resource identifiers link production data to equipment

events [33]; ISA-95 distinguishes manufacturing operations management from equipment monitoring and control [34]. The present decision function takes batch, recipe, and equipment-state inputs and generates an offline event schedule. Commercial PECVD hardware provides the process context: Cygnus Optima Prime uses modular process chambers and automated wafer handling [35], while Fig. 2 shows an open PECVD chamber with its showerhead and substrate-support region [36]. The dual-source topology used for scheduling is specified in Fig. 1.

The implemented offline workflow comprises configuration and PW construction, two-stage optimization, and validation and event export. Batch data specify wafer identities, sources, and preset modes; equipment data specify chambers, robots, load locks, action times, residency bounds, cleaning intervals, and dummy inventory. The constructor creates product, dummy, or mixed PWs and establishes full-occupancy and cleaning demands. Two-stage MILP, SPBS, and learning-enhanced SCIP search then generate a finite-batch schedule. Material identities persist through pairing, handling, and return; resource identities connect configuration, conflict constraints, and event output.

The proposed integration interface uses material and resource identifiers to associate planned actions with resource states and completion events. These fields support subsequent controller mapping and remaining-task rescheduling. The computational study evaluates offline schedules generated from initial batch states and prescribed parameter perturbations.

Inputs comprise (1) batches, wafer identities, material sources, and modes; (2) recipe durations, residency bounds, cleaning rules, topology, capacities, and action times; (3) initial equipment positions, dummy inventory, and resource availability; and (4) total and stage budgets and termination conditions. Each output event specifies its identifier, wafer or PW, resource, origin and destination, planned start and finish, predecessor, chamber, and mode. The output also reports $C_{\max}$, waiting and cadence measures, incumbent objective value, dual bound, terminal gap, and feasibility and optimality status.

After validation against the frozen model, the schedule is exported as timed actions covering LP release and return, ATR handling, AL alignment, load-lock loading, pumping, venting and unloading, VTR handling, chamber rotation, processing, and cleaning. Each record includes its action type, resource, wafer or PW, origin and destination, planned time, and duration. Deployment requires an equipment adapter that maps these fields to tool-specific protocols and checks controller interlocks and execution timing. The exported records and validation status form the input to this adapter.

2.2. *PW-pair modeling for the two process modes*

The equipment cell is represented as a three-layer mapping from configuration inputs through operating decisions to execution outputs. Inputs include product wafer counts and modes, material-source interfaces, equipment action times, cleaning intervals, residency limits, load-lock capacity, and dummy inventory. The operating decisions jointly determine product and dummy routes, the start and finish of every physical action, and the order of shared-resource use. The output is a complete



Figure 2: Open chamber of a commercial PECVD system, showing the showerhead and substrate-support region. Photograph from Syskey Technology Co., Ltd. [36]. The photograph illustrates PECVD hardware; Fig. 1 defines the modeled equipment topology.

finite-batch timeline together with its feasibility and optimality status. The boundary covers the product path from an LP through processing and back to an LP and the closed dummy loop from vacuum storage through filling or cleaning and back to storage. No buffer outside the LP, aligner, load locks, rotating chambers, and dummy storage is assumed; every wait therefore corresponds to an explicit physical-occupancy interval.

A wafer pair (PW pair) is a physical two-wafer transport and chamber-occupancy unit. Its members may be product+product (product PW pair), dummy+dummy (dummy PW pair), or product+dummy (mixed PW pair). Product wafers are preferentially paired by mode and index; an odd tail product may be paired with one dummy, while full-occupancy filling and cleaning may use dummy PW pairs. The sets $\mathcal{P}^F$ and $\mathcal{P}^L$ index product-containing PW pairs, including product PW pairs and any required mixed PW pairs. Alignment and residency remain wafer-level operations; the two members share a time window only on a selected two-hand transfer or the same chamber side. Preprocessing specifies product-containing PW pairing and compatible chamber positions. The MILP optimizes the remaining activation and assignment decisions, physical dummy identities, paired transfers, resource order, and event times within this layout. Legal cleaning segments are generated from the cleaning interval and complete recipe boundaries, after which the MILP activates the cleaning tasks and schedules their times.

A product follows LP–ATR–AL–ATR–upper load lock–VTR–rotary chamber–VTR–lower load lock–ATR–LP: the ATR first places the wafer on the aligner, then retrieves it for transfer into the vacuum domain. A dummy travels from vacuum storage through the VTR to the chamber and returns through the VTR after use. A regular all-product 4×1 batch contains two product PW pairs, hence four product wafers, and processes all four positions synchronously once before last-in–

first-out unloading; only the last incomplete batch may use a dummy or mixed PW pair to fill all positions. In 2×2, every product-containing PW pair occupies two adjacent process cycles so that each of its product members is processed twice. For three product PW pairs, a dummy PW pair D is loaded first, the chamber rotates, and P1 is loaded before D+P1 undergoes chamber processing. Chamber processing then alternates with "chamber rotation + chain slide", changing the occupancy successively to P1+P2, P2+P3, and P3+D; P1, P2, and P3 are each processed twice before all wafers are unloaded. Figures 3 and 4 show the event sequences.

Tables 1 and 2 list every symbol in the core formulation and distinguish sets/indices, input constants, fixed binary aliases, binary decisions, continuous decisions, and derived quantities. In the implementation, x^F and x^L are continuous $[0, 1]$ aliases fixed to zero or one by equality; this preserves uniform export names without creating extra branching decisions.

2.3. *Source interfaces, operation activation, and time relations*

The fixed incidence parameter satisfies

$$\sum_{s \in \mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{1}$$

Equation (1) assigns each material unit to exactly one source. Setting $|\mathcal{S}| = 1$ gives the single-source tool; using two interfaces and assigning products and dummies to different interfaces gives the dual-source tool.

If $\mathcal{A}^{\text{int}}$ is the internal network and $\mathcal{A}_s$ contains release and return arcs for source s, then

$$\mathcal{A} = \mathcal{A}^{\text{int}} \cup \left(\bigcup_{s \in \mathcal{S}} \mathcal{A}_s \right). \tag{2}$$

Equation (2) means that one- and two-source tools differ only at route endpoints; processing, cleaning, shared resources, and timing use the same internal model, so a dual-source tool is not the sum of two independent single-source schedules.

Canonical pair positions satisfy

$$\begin{aligned} x^F_{pcbl} &= \bar{x}^F_{pcbl}, \quad \sum_{c,b,l} x^F_{pcbl} = 1 \quad (p \in \mathcal{P}^F), \\ x^L_{pck} &= \bar{x}^L_{pck}, \quad \sum_{c,k} x^L_{pck} = 1 \quad (p \in \mathcal{P}^L). \end{aligned} \tag{3}$$

Equation (3) assigns each wafer pair to one compatible position and fixes equivalent locations to canonical indices. Event times and resource order remain optimization decisions within the prescribed layout.

A position containing a pair must be active, and candidate positions are used consecutively:

$$\begin{aligned} x^F_{pcbl} &\le u^F_{cb}, \quad u^F_{c,b+1} \le u^F_{cb}, \\ x^L_{pck} &\le u^L_{ck}, \quad u^L_{c,k+1} \le u^L_{ck}. \end{aligned} \tag{4}$$

Equation (4) activates any batch or chain position containing a product and excludes a nonphysical inactive position followed by an active one.

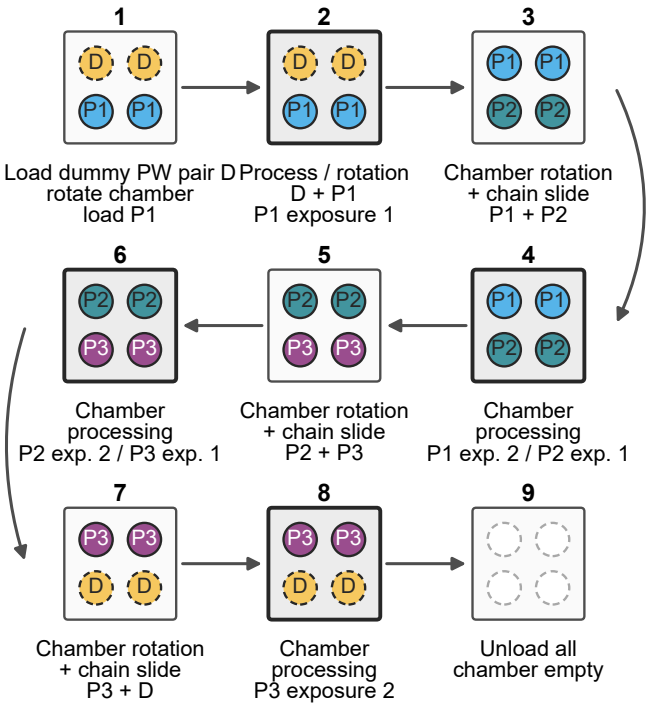


Figure 3: 2×2 chained processing. A PW pair may be P+P, D+D, or P+D; here each Pi is a P+P product PW pair and D is a D+D dummy PW pair. Step 1 loads D, rotates the chamber, and loads P1; Step 2 performs chamber processing on D+P1; Step 3 applies chamber rotation + chain slide to obtain P1+P2. Repeating "chamber processing–chamber rotation + chain slide" makes P1, P2, and P3 participate in two adjacent process cycles, so every product wafer is processed twice before all wafers are unloaded.

For each candidate operation o, activation and duration satisfy

$$0 \le s_o \le M b_o, \qquad 0 \le e_o \le M b_o, \qquad e_o = s_o + d_o b_o. \tag{5}$$

Equation (5) sets inactive operation times to zero and gives an active operation duration d_o. It applies to robot moves, alignment, pump/vent, rotation, product processing, dummy transport, and cleaning.

For every required process arc $(o, o') \in \mathcal{A}$,

$$b_{o'} \le b_o, \qquad s_{o'} \ge e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \tag{6}$$

Equation (6) activates a successor only with its predecessor and, when both exist, starts it after the predecessor and minimum lag. These arcs connect product routes, dummy routes, and load–process–unload chains.

Table 1: Core notation I: sets, indices, and input parameters

Symbol	Domain and role	Meaning
$\mathcal{S}, \mathcal{I}$	sets/indices	Material-source interfaces and material units; $i \in \mathcal{I}$ may be a product or dummy demand.
$\mathcal{W}$	set/index	Product wafers, indexed by w.
$\mathcal{P}^F, \mathcal{P}^L$	sets/indices	Product-containing PW pairs assigned to 4×1 and 2×2; they include P+P product pairs and any required P+D mixed pairs, indexed by p.
C	set/index	Rotary process chambers, indexed by c.
$\mathcal{B}_c, \mathcal{K}_c$	ordered sets/indices	Candidate 4×1 batches b and 2×2 product-chain positions k in chamber c.
$\mathcal{L} = \{1, 2\}$	set/index	Front and back sides of a 4×1 batch.
$\mathcal{Q}_c, \mathcal{E}_c$	ordered sets/indices	2×2 process cycles q and cleaning segments e in chamber c.
$\mathcal{V}_c$	set/index	Union of all 4×1 batches and 2×2 cycles in chamber c.
$O, \mathcal{A}$	sets/indices	Candidate physical operations and required process-precedence arcs.
$\mathcal{R}, \Lambda_r$	sets/indices	Shared resources and physical slots of resource r.
$\mathcal{J}_c^D, \mathcal{T}_c$	sets/indices	Dummy-wafer demands and reusable physical dummies of chamber c.
χ_{is}	known binary constant	Whether material unit i belongs to source interface s.
$\bar{x}^F_{pcbl}, \bar{x}^L_{pck}$	known binary constants	Canonical pair-to-4×1-side or pair-to-2×2-position assignment.
$q_c^-(k), q_c^+(k)$	known nonnegative integer maps	First and second process cycles of 2×2 chain position k, with $q_c^+(k) = q_c^-(k) + 1$.
$\bar{z}_{ce}, \bar{a}^F_{cbe}, \bar{a}^L_{cqe}$	known binary constants	Active cleaning segments and pre-generated membership of batches or cycles.
$H, \bar{Q}_c$	nonnegative integer constants	Maximum cycles between cleanings and generated 2×2 cycles in chamber c.
$d_o, \delta_{oo'}$	nonnegative constants	Duration of operation o and minimum lag on arc (o, o').
d_ν^F, d^L, r_c	nonnegative constants	4×1 stage duration, 2×2 process time, and chamber half-rotation time.
$\rho^r_{oo'}$	nonnegative constant	Empty travel or reset time from operation o to o' on resource r.
$M, \bar{R}_{wr}$	positive constants	Valid horizon-based big-M and maximum residency of wafer w in restricted region r.
κ_o, Γ_o	integer/set constants	Slots required by operation o and the slots it may use.
$\lambda_1, \ldots, \lambda_4$	nonnegative constants	Stage-two weights for total wait, maximum wait, cadence deviation, and soft-limit excess.
$\bar{\omega}_\alpha, \varepsilon_C$	nonnegative constants	Soft limit for wait α and the stage-two makespan tolerance.
$\mathcal{F}, \mathcal{D}_{\text{fix}}$	sets	Full MILP feasible region and discrete indices fixed in stage two.
$\mathcal{Q}^w, \mathcal{I}_h$	sets/indices	Wait quantities and process starts admitting cadence second differences.

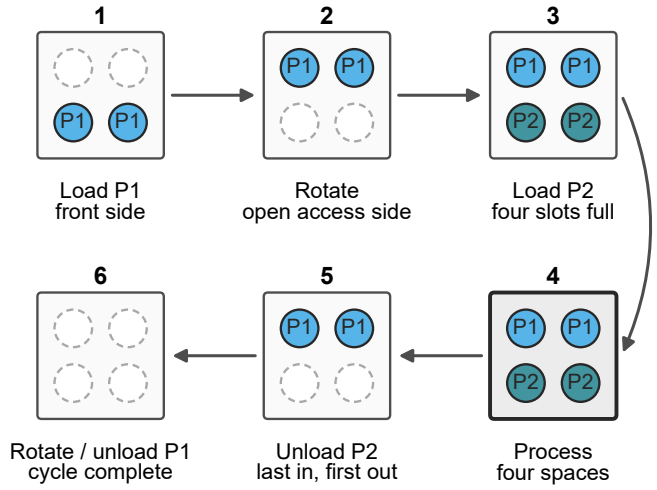


Figure 4: 4×1 parallel processing. A regular all-product batch uses two product PW pairs, comprising four product wafers, and receives one synchronous full-occupancy process before last-in–first-out unloading; a dummy or mixed PW pair can complete the final short batch.

2.4. Occupancy and event order in the two processing modes

For 4×1, let $A^F_{cb} = \sum_{p,l} x^F_{pcbl}$ be the number of product pairs assigned to batch (c, b). Full occupancy and dummy filling are

$$\begin{aligned} A^F_{cb} &= \sum_{p \in \mathcal{P}^F} \sum_{l \in \mathcal{L}} x^F_{pcbl}, \\ \sum_{p \in \mathcal{P}^F} x^F_{pcbl} + f^F_{cbl} &= u^F_{cb} \qquad (l \in \mathcal{L}), \\ \sum_{l \in \mathcal{L}} f^F_{cbl} &= 2u^F_{cb} - A^F_{cb}. \end{aligned} \tag{7}$$

Equation (7) gives each active 4×1 batch two sides, each occupied by one product pair or one dummy pair; all four chamber positions are therefore filled at process start.

Active batches are consecutive and dummy filling is restricted to the last active batch:

$$\begin{aligned} A^F_{cb} &\geq u^F_{cb}, & u^F_{c,b+1} &\leq u^F_{cb}, \\ A^F_{cb} &\geq 2u^F_{c,b+1}, & t^F_{cb} &\leq u^F_{cb}, \\ t^F_{cb} &\leq 1 - u^F_{c,b+1}, & t^F_{cb} &\geq u^F_{cb} - u^F_{c,b+1}. \end{aligned} \tag{8}$$

Equation (8) requires at least one product pair in an active batch and two product pairs if a later batch is active. The last three inequalities set $t^F_{cb} = 1$ exactly at the active tail; the final candidate uses $t^F_{cB} = u^F_{cB}$.

Let FL, BL, P, BU, FU denote front load, back load, processing, back unload, and front unload. Then

$$\begin{aligned} e^{F,\nu}_{cb} &= s^{F,\nu}_{cb} + d^F_\nu u^F_{cb} \qquad (\nu \in \{FL, BL, P, BU, FU\}), \\ s^{F,BL}_{cb} &\geq e^{F,FL}_{cb} + r_c - M(1 - u^F_{cb}), \\ s^{F,P}_{cb} &= e^{F,BL}_{cb}, \\ s^{F,BU}_{cb} &\geq e^{F,P}_{cb} - M(1 - u^F_{cb}), \\ s^{F,FU}_{cb} &\geq e^{F,BU}_{cb} + r_c - M(1 - u^F_{cb}). \end{aligned} \tag{9}$$

Equation (9) loads the front side, rotates to load the back, starts synchronous processing after the back load, and then unloads the back side before rotating and unloading the front. Consecutive batches are linked by $s^{F,FL}_{c,b+1} \geq e^{F,FU}_{cb} - M(1 - u^F_{c,b+1})$.

For 2×2, each product pair occupies one ordered chain position:

$$u^L_{ck} = \sum_{p \in \mathcal{P}^L} x^L_{pck}, \qquad u^L_{c,k+1} \leq u^L_{ck}, \qquad a^L_c = u^L_{c1}. \tag{10}$$

Equation (10) creates a one-to-one relation between product pairs and consecutive chain positions and uses a^L_c to indicate whether chamber c contains a 2×2 chain.

Table 2: Core notation II: decision variables, fixed aliases, and derived quantities

Symbol	Domain and role	Meaning
x^F_{pcbl}, x^L_{pck}	fixed continuous $[0,1]$ aliases	Product-containing PW-pair placement on a 4×1 batch side or at a 2×2 chain position.
μ^L_{pcq}	fixed continuous $[0,1]$ alias	Whether 2×2 product-containing PW pair p is processed in cycle q of chamber c.
$u^F_{cb}, f^F_{cbl}, t^F_{cb}$	binary variables	4×1 batch activation, dummy filling on a side, and final active batch.
$u^L_{ck}, a^L_c, v^L_{cq}, h^L_{ck}$	binary variables	2×2 position, chain, process-cycle, and chain-tail activation.
$z_{ce}, a^F_{cbe}, a^L_{cqe}, q^{\mathrm{cl}}_{ce}$	fixed-binary aliases/variables	Cleaning segments, task membership, and cleaning-task activation.
τ_{ce}	fixed binary parameter	Whether segment e is the final production segment and exactly exhausts H cycles, thereby triggering terminal cleaning.
$q_j, g_{j\tau}$	binary variables	Dummy demand and assignment to physical dummy τ.
$b_o, y^r_{oo'}$	binary variables	Operation activation and whether o precedes o' on resource r.
$h_{o\ell}$	binary slot variable	Whether operation o occupies physical slot ℓ.
γ_{pa}	binary variable	Whether pair p uses one paired transfer on robot route a.
s_o, e_o	nonnegative variables	Start and end times of operation o.
$s^{F,v}, e^{F,v}, s^L, e^L$	continuous aliases	Stage-specific notation for s_o, e_o in the two modes.
A^F_{cb}, N_{ce}	derived continuous/integer quantities	Product-pair count in a 4×1 batch and active cycles in a cleaning segment.
a_{vce}	fixed binary alias	Whether task $v\in\mathcal{V}_c$ belongs to cleaning segment e.
$C_w, C_{\max}$	nonnegative variables	Product completion time and lot makespan.
$\omega_\alpha, L_{\mathrm{sum}}, L_{\max}$	nonnegative variables/derived quantities	Stage-two wait or idle quantities, their sum, and their maximum.
$\Delta_{ceq}, v_{ceq}, \eta_\alpha$	derived/nonnegative variables	Cadence second difference, its absolute-value bound, and soft wait excess.
h_{ceq}, Φ	continuous alias/derived objective	Start of the qth process in a segment and stage-two schedule-quality objective.
$\xi, \widehat{\xi}, \widehat{C}$	vector/stage constants	Current variables, stage-one incumbent, and incumbent makespan.

Let $q^-_c(k)$ and $q^+_c(k)$ denote the first and second process cycles of chain position k. The pair–cycle incidence is

$$\begin{aligned} q^+_c(k) &= q^-_c(k)+1,\\ \mu^L_{pcq} &= \sum_{\substack{k\in\mathcal{K}_c:\\ q\in\{q^-_c(k),q^+_c(k)\}}} x^L_{pck},\\ \sum_{q\in\mathcal{Q}_c}\mu^L_{pcq} &= 2\sum_{k\in\mathcal{K}_c} x^L_{pck}. \end{aligned} \tag{11}$$

Equation (11) maps every assigned 2×2 product-containing PW pair to exactly two adjacent process cycles. Every product member inherits the same μ^L_{pcq} incidence, so a product is not omitted even when an odd tail uses a P+D mixed PW pair. The equation also distinguishes the modes explicitly: a 2×2 product-containing pair spans two cycles, whereas two 4×1 side pairs are processed once in one full batch.

The generated layout supplies the number $\bar{Q}_c$ of process cycles, and h^L_{ck} identifies the chain tail:

$$\begin{aligned} v^L_{cq} &= \begin{cases} a^L_c, & q\le\bar{Q}_c,\\ 0, & q>\bar{Q}_c, \end{cases}\\ \sum_k h^L_{ck} &= a^L_c, \qquad h^L_{ck}\le u^L_{ck}, \qquad h^L_{ck}+u^L_{c,k+1}\le 1,\\ h^L_{ck} &\ge u^L_{ck}-u^L_{c,k+1}. \end{aligned} \tag{12}$$

Equation (12) gives $n+1$ process cycles for a segment with n product-containing PW pairs: the head is dummy PW pair+P1, internal cycles are Pi+P$i+1$, and the tail is Pn+dummy PW pair. The final relations make h^L_{ck} one only at the last active product position.

For a legal chain segment $(q_0,\dots,q_T)$, let H_0, B_q, and T_0 denote head loading, internal exchange, and tail unloading. Then

$$\begin{aligned} e^L_{cq}-s^L_{cq} &= d^L v^L_{cq},\\ s^L_{cq_0} &= e^{H_0}_c,\\ s^B_{cq} &\ge e^L_{c,q-1}-M(1-a^L_c), \qquad s^L_{cq}=e^B_{cq} \quad (q_0<q\le q_T),\\ s^{T_0}_c &\ge e^L_{cq_T}-M(1-a^L_c). \end{aligned} \tag{13}$$

Equation (13) begins the first process immediately after head loading, performs an internal exchange only after the previous cycle, and unloads the tail only after the last process. It directly realizes dummy+P1, P1+P2, …, Pn+dummy.

2.5. Cleaning and reusable dummy wafers

Cleaning segments are generated from H and complete process boundaries and enter the MILP as fixed binary aliases:

$$z_{ce}=\bar{z}_{ce}, \qquad a^F_{cbe}=\bar{a}^F_{cbe}u^F_{cb}, \qquad a^L_{cqe}=\bar{a}^L_{cqe}v^L_{cq}. \tag{14}$$

Equation (14) prevents arbitrary cleaning locations: 4×1 may split only at batch boundaries and 2×2 only after a complete chain segment, while the MILP still schedules process and cleaning times.

Let N_{ce} be the number of active process cycles in segment e:

$$\begin{aligned} N_{ce} &= \sum_b a^F_{cbe}+\sum_q a^L_{cqe}, \qquad z_{ce}\le N_{ce}\le Hz_{ce},\\ q^{\mathrm{cl}}_{ce} &= z_{c,e+1}+\tau_{ce}, \qquad z_{c,E_c+1}=0. \end{aligned} \tag{15}$$

Equation (15) places at least one and at most H process cycles in an active segment. It activates an internal cleaning before every subsequent production segment and, when the final segment exactly exhausts H cycles, sets $\tau_{ce}=1$ to activate terminal cleaning. Both are complete equipment actions; only the terminal action has no subsequent product processing.

Let $\mathcal{V}_c$ be the union of 4×1 batches and 2×2 cycles and let a_{vce} take the corresponding a^F or a^L. Cleaning order and the physical cleaning chain satisfy

$$\begin{aligned} s_v &\geq e^{\mathrm{cl}}_{c,e-1} - M(1 - a_{vce}) && (e > 1), \\ e_v &\leq s^{\mathrm{cl}}_{ce} + M(1 - a_{vce}) + M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BL}}_{ce} &\geq e^{\mathrm{cl,FL}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,P}}_{ce} &\geq e^{\mathrm{cl,BL}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BU}}_{ce} &\geq e^{\mathrm{cl,P}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,FU}}_{ce} &\geq e^{\mathrm{cl,BU}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}). \end{aligned} \tag{16}$$

Equation (16) starts a segment after the preceding cleaning and finishes it before the next cleaning. A cleaning task loads both chamber sides with dummies, rotates, processes, and unloads in reverse, thereby occupying the VTR, chamber, and four physical dummies rather than adding virtual downtime.

Dummy demand j is activated by a physical event:

$$q_j = \begin{cases} f^F_{cbl}, & j \text{ is a final-} 4 \times 1 \text{ filling demand}, \\ a^L_c, & j \text{ is a } 2 \times 2 \text{ head or tail demand}, \\ q^{\mathrm{cl}}_{ch}, & j \text{ is a cleaning demand}. \end{cases} \tag{17}$$

Equation (17) prevents dummies from becoming an unlimited abstract filler: a physical demand exists only for a final batch, chain head/tail, or active cleaning task.

Each active demand selects one physical dummy from inventory:

$$\sum_{\tau \in \mathcal{T}_c} g_{j\tau} = q_j, \qquad j \in \mathcal{J}^D_c. \tag{18}$$

Equation (18) assigns every active demand exactly once. Mutual exclusion subsequently orders repeated uses of the same physical dummy, so it can be reused only after unloading and return to storage.

2.6. Robots, load locks, chambers, and time resources

ATR, AL, VTR, and each rotary chamber are unary resources; the upper and lower load locks each have two physical slots; each dummy in inventory is a reusable slot. An operation o requiring κ_o slots satisfies

$$\sum_{\ell \in \Gamma_o} h_{o\ell} = \kappa_o b_o. \tag{19}$$

Equation (19) assigns an active operation the required number of legal physical slots and assigns none to an inactive operation. AL, ATR, VTR, and a chamber have one slot, while load locks and dummy inventory retain physical identities.

If operations o and o' may compete for slot ℓ of resource r, then

$$\begin{aligned} s_{o'} &\geq e_o + \rho^r_{oo'} - M(1 - y^r_{oo'}) - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}), \\ s_o &\geq e_{o'} + \rho^r_{o'o} - M y^r_{oo'} - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}). \end{aligned} \tag{20}$$

Equation (20) makes $y^r_{oo'}$ choose one order whenever two actions use the same slot and includes empty robot travel or equipment reset through $\rho^r_{oo'}$. It applies to ATR, AL, both load locks, VTR, rotary chambers, and individual dummies, preventing concurrent robot actions and reuse before release.

An individual product or dummy stage must coincide with its selected batch, chain position, or cleaning window. For selection x and reference stage $[s^{\mathrm{ref}}, e^{\mathrm{ref}}]$,

$$\begin{aligned} -M(1 - x) \leq s_o - s^{\mathrm{ref}} \leq M(1 - x), \\ -M(1 - x) \leq e_o - e^{\mathrm{ref}} \leq M(1 - x). \end{aligned} \tag{21}$$

Equation (21) synchronizes the individual action with its physical stage when $x = 1$ and releases the candidate relation when $x = 0$, connecting pair positions, product routes, dummy demands, and chamber windows.

If pair p uses one paired transfer on robot route a, the two wafer stages are synchronized:

$$\begin{aligned} -M(1 - \gamma_{pa}) \leq s_{w_1 a} - s_{w_2 a} \leq M(1 - \gamma_{pa}), \\ -M(1 - \gamma_{pa}) \leq e_{w_1 a} - e_{w_2 a} \leq M(1 - \gamma_{pa}). \end{aligned} \tag{22}$$

Equation (22) means that $\gamma_{pa} = 1$ moves a complete pair in one non-preemptive robot action. If $\gamma_{pa} = 0$, the wafers still occupy the same robot separately and are ordered by Eq. (20).

For the entry and exit events of wafer w in restricted equipment region r,

$$0 \leq s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \leq \overline{R}_{wr}. \tag{23}$$

Equation (23) limits the physical stay in an upper load lock, chamber, lower load lock, or restricted transport region. Load-lock occupation also spans placement, pump/vent, opposite-side pickup, and pressure reset, so placement by a robot is not mistaken for slot release.

2.7. Makespan-first two-stage optimization

Let C_w be the time at which product wafer w completes the lower-load-lock return and reaches its load port. Stage one solves

$$C_w = e_{w,\mathrm{return}}, \qquad C_{\max} \geq C_w \ (w \in \mathcal{W}), \qquad \min_{\xi \in \mathcal{F}} C_{\max}. \tag{24}$$

Equation (24) measures makespan at the last product wafer's return to LP after its complete end-to-end route. Internal cleaning affects $C_{\max}$ through resource occupancy and precedence that delay subsequent product returns. Terminal cleaning remains in the action sequence, but its completion time does not directly lower-bound $C_{\max}$. It can still affect other chambers' product returns through competition for the shared VTR; this indirect effect is determined by the complete schedule. WPH uses product-return makespan as its denominator. The feasible region $\mathcal{F}$ is defined by Eqs. (1)–(23) expanded over all applicable indices.

Given stage-one incumbent $\widehat{\xi}$ and makespan $\widehat{C}$, stage two fixes the discrete structure indexed by $\mathcal{D}_{\mathrm{fix}}$ and minimizes linear

schedule quality within the makespan tolerance:

$$\begin{aligned}
\xi_d &= \widehat{\xi}_d \quad (d \in \mathcal{D}_{\mathrm{fix}}), \qquad C_{\max} \le \widehat{C} + \varepsilon_C, \\
L_{\mathrm{sum}} &= \sum_{\alpha \in Q^w} \omega_\alpha, \qquad L_{\max} \ge \omega_\alpha, \\
\Delta_{ceq} &= (h_{ce,q+1} - h_{ceq}) - (h_{ceq} - h_{ce,q-1}), \\
v_{ceq} &\ge \Delta_{ceq}, \qquad v_{ceq} \ge -\Delta_{ceq}, \\
\eta_\alpha &\ge \omega_\alpha - \bar{\omega}_\alpha, \qquad \eta_\alpha \ge 0, \\
\min \Phi &= \lambda_1 L_{\mathrm{sum}} + \lambda_2 L_{\max} + \lambda_3 \sum_{(c,e,q) \in \mathcal{I}_h} v_{ceq} + \lambda_4 \sum_{\alpha \in Q^w} \eta_\alpha .
\end{aligned} \tag{25}$$

Equation (25) preserves the stage-one makespan bound and fixes the principal discrete routes and resource orders. Retained paired-transfer choices and continuous times can change to reduce total wait, maximum wait, and variation between consecutive processing intervals within each segment. Optional soft-limit excess can also be penalized. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$. With the soft waiting limit disabled, no excess variables are generated. For a time-limited first stage, the best incumbent value $\widehat{C}$ supplies the second-stage bound; variables in $\mathcal{D}_{\mathrm{fix}}$ are fixed while designated paired-transfer variables remain adjustable.

2.8. Coverage and model boundary

After index expansion, Eqs. (1)–(25) jointly describe the finite-batch dual-source problem with mixed $4 \times 1/2 \times 2$ operation, periodic cleaning, and finite dummy inventory. Equations (1)–(2) encode source interfaces: one interface is single source, while two interfaces with products and dummies assigned separately are dual source. Equations (3)–(6) encode locations, activation, durations, and precedence for product-containing PW pairs. Equations (7)–(9) encode a 4×1 full batch formed by two chamber-side PW pairs, final-batch filling, rotation, and last-in–first-out unloading. Equations (10)–(13) encode the consecutive 2×2 chain, two adjacent exposures per product-containing PW pair, dummy-PW-pair head/tail filling, and internal exchange. Equations (14)–(16) encode legal cleaning boundaries, segment capacity, and the physical cleaning chain. Equations (17)–(18) encode dummy demand and finite inventory. Equations (19)–(23) encode mutual exclusion of robots, load locks, chambers, and dummies, paired transfer, and residency. Equations (24)–(25) encode end-to-end makespan followed by wait/cadence refinement. A pure mode is obtained by making the other mode set empty.

Every active material route has a finite precedence chain from release to return, and every pair of operations competing for one resource must be ordered. If those inequalities formed a directed cycle, summing them would require an event to occur later than itself by a positive duration, so the combination is infeasible. The resource-ordering and process-precedence constraints therefore eliminate circular waits from the schedule.

Wafer pairing, chamber-side placement, and cleaning segmentation are prescribed inputs. Optimization determines the remaining activation, assignment, transfer, ordering, and timing decisions within that layout. Mode comparisons therefore apply to the specified PW layouts and deterministic action times. Random failures and online rescheduling are outside the evaluated setting.

3. Structure-aware MILP solution framework

The model in Section 2 must return an executable equipment timeline within a given decision window, rather than only an objective value. On the primal side, full occupancy, dual-source logistics, and shared-resource order can delay the first competitive feasible plan. On the dual side, generic cut scores do not reveal whether a cut mainly tightens discrete choices, continuous event times, or their coupling. We first construct a feasible starting point with a structure-preserving binary skeleton (SPBS), and then use RDMCT-HBS to select cuts in SCIP separation callbacks. SPBS improves finite-time plan availability, and RDMCT-HBS controls cut counts and insertion order, while SCIP retains responsibility for feasibility checks, valid-cut generation, bounds, and optimality proofs [37, 38].

The complete workflow comprises six stages: construct and validate an initial solution, generate candidate cuts, predict the cut count, search for an ordered cut set, complete its structure, and resume exact search. The learning component neither generates inequalities nor changes candidate-cut coefficients; it determines only the retained prefix and input order of rows already validated by SCIP. The learned policy operates at the search-control layer; SCIP validates schedules and certifies optimality.

3.1. SPBS initial solution

SPBS first constructs a discrete skeleton from the recipe and equipment layout, then solves for event times in a restricted MILP. Product wafers are paired in input order. The 4×1 pairs are mapped sequentially to chambers, batches, and front/back positions, whereas the 2×2 pairs are assigned round-robin to chamber-chain positions. This determines batch-use, tail-batch, used-position, and last-position indicators. Cleaning counts partition each chamber sequence into process segments: 4×1 batches are assigned to their cleaning epochs, while a 2×2 chain ends its current segment with process-enabling carriers (PECs) when the cleaning threshold is reached and resumes after cleaning with a new PEC-bounded head. The skeleton therefore determines what is processed, in which chamber, and at which legal cleaning boundary; detailed PEC occupancy and all continuous event times remain governed by the MILP.

At the resource level, SPBS derives a conservative order from the same physical route. ATR first schedules 4×1 inputs and returns across chambers and stages the first 2×2 pair of a chamber immediately after that chamber's final full batch. It then advances chain inputs and returns segment by segment. Deterministic odd/even load-lock slots break slot symmetry, and the VTR order is aligned with ATR batch progression and cleaning actions. Let $\mathcal{F}_q$ be the binary variables fixed in relaxation profile q, with skeleton value $\widehat{x}_j$. The temporary model imposes

$$\ell_j^{(q)} = u_j^{(q)} = \widehat{x}_j, \qquad j \in \mathcal{F}_q \tag{26}$$

Equation (26) fixes only the binary choices regarded as reliable in the current profile. Start, completion, and waiting times, together with resource orders outside that profile, remain free.

If the strict skeleton does not yield a feasible solution within its budget, SPBS enlarges the search space through five implemented profiles. First, `spbs_full` fixes product/batch assignment, activation, cleaning epochs, load-lock slots, ATR/AL/LL order, and VTR order. Second, `spbs_relaxed_vtr` releases the high-risk VTR order. Third, `spbs_relaxed_ll` additionally releases load-lock order. Fourth, `spbs_fixed_slots` releases ATR, AL, and all remaining resource orders but retains structural assignments and slots. Fifth, `spbs_structure_only` also releases slots, leaving only assignment, activation, and cleaning structure fixed. For instances of at least 32 wafers, the implementation begins from fixed-slot or structure-only profiles so that more time is available for feasibility search; under a tight waiting-time cap it also retains a fully unfixed `spbs_unrestricted` attempt.

Every profile rebuilds a temporary model, sets the objective to zero, uses feasibility emphasis, aggressive heuristics, fast presolve, no separation, and stops at the first solution. Once a physical schedule is found, the program reconstructs the full model, fixes its principal discrete and continuous values, releases only secondary stability auxiliaries, completes them, and invokes the complete SCIP solution check. Only a checked high-precision solution file is passed to the production model as an incumbent. No SPBS bound is transferred, so the production solve remains free to improve the schedule and complete the global proof. In the formal benchmark SPBS is precomputed offline; its metadata records the accepted profile and all attempts, and a SHA-256 hash of the serialized LP prevents a start from being reused on a different model. Its construction cost is excluded from reported PDI and main-solve time.

3.2. HEM baseline and role-aware representation

Prior learning-based cut selectors address generic candidate sets with reinforcement learning, learned ranking, and look-ahead imitation [29–31]. HEM further identifies three coupled decisions—which cuts, how many cuts, and in what order—and handles them with a high-level count policy and a low-level sequence policy [32]. We retain HEM's hierarchical decomposition and pointer decoding and use its 13D implementation as the direct baseline. Training instances, budgets, random seeds, candidate limits, and hidden dimensions are shared while representation, decoding, and post-selection completion are compared. HEM was subsequently extended to hierarchical sequence/set models [39]; HEM-13D here is a controlled reproduction of the 2023 sequence implementation.

The 13D HEM candidate vector comprises objective parallelism, efficacy, normalized support, integral-support ratio, normalized violation, the mean/maximum/minimum/standard deviation of row coefficients, and the corresponding four objective-coefficient statistics. These features describe immediate LP quality and numerical scale. They do not distinguish two similarly efficacious rows when one acts on discrete route choices and the other on continuous event times. In the present model, that distinction corresponds directly to changing a process/resource combination versus tightening the timing of an existing combination.

Starting from the 13 generic HEM features, variables are partitioned by solver metadata into binary, general integer, objective-active continuous, continuous auxiliary, and other roles. The coefficient mass of role g in cut i is

$$p_{ig} = \frac{\sum_{j:x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{27}$$

Equation (27) is the fraction of absolute cut-coefficient mass carried by a mathematical variable role. Beyond the five role masses, the added features are mass on variables with two finite global bounds, positive-coefficient mass, discrete–continuous coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$, the largest non-residual role mass, and normalized role entropy. The resulting vector has 23 dimensions. Roles are identified solely from SCIP variable types, objective coefficients, and global bounds; neither variable names nor equipment labels are read, so variable renaming leaves the representation unchanged.

HEM uses a hierarchical "count policy + ordered-subset policy" decomposition: its high level predicts the number of cuts from a sequence representation, while its low level uses an LSTM encoder–decoder with pointer attention to select cuts sequentially [32, 40]. Building on this hierarchy, the proposed selector introduces a role-aware grouped gated encoder before the LSTM, a role-submodular completion step after the pointer sequence, and width-three beam search during formal inference, which together form the complete RDMCT-HBS selection pipeline.

3.3. Role-aware encoding and hierarchical sequence decoding

Let m_t be the number of candidates retained at separation callback t. For candidate cut i, write the 23D input as $x_i = [x_i^H; x_i^R]$, where $x_i^H \in \mathbb{R}^{13}$ contains the generic HEM features and $x_i^R \in \mathbb{R}^{10}$ contains the proposed role-aware features. The full vector is first layer-normalized; the two feature groups are then projected separately into the common hidden space and fused by an elementwise gate generated from all 23 features:

$$\begin{aligned} \widetilde{x}_i &= \mathrm{LN}(x_i), \\ u_i^H &= W_H \widetilde{x}_i^H + b_H, \qquad u_i^R = W_R \widetilde{x}_i^R + b_R, \\ g_i &= \sigma(W_g \widetilde{x}_i + b_g), \qquad z_i = g_i \odot u_i^H + (1 - g_i) \odot u_i^R. \end{aligned} \tag{28}$$

Here $z_i \in \mathbb{R}^{128}$ is the fused candidate embedding. Unlike a single projection of the 23D vector, the two branches preserve generic LP quality and mathematical-role structure separately, while g_i determines their candidate-specific contribution in each hidden dimension. Thus two cuts with similar efficacy but different concentration on discrete route variables or continuous timing variables can remain distinguishable. This grouped gated adapter is a structural change to HEM's input representation introduced in this work.

The fused candidates, in the deterministic order established by Step 1, are passed through a single-layer LSTM encoder:

$$(h_i, c_i) = \mathrm{LSTM}(z_i, h_{i-1}, c_{i-1}), \qquad i = 1, \ldots, m_t. \tag{29}$$

The states h_i turn local cut descriptors into context-dependent candidate representations and are used by both the high-level retained-count policy and the low-level ordered selector. A 128D hidden state is used to encode the ordered candidates into a contextual sequence representation. Fixing the input order also makes the encoded sequence reproducible for an identical candidate pool.

At low-level decoding step r, the decoder state d_r and all encoder states h_i are used for one glimpse attention followed by pointer attention. Let $M_{ri} = -\infty$ if candidate i already appears in the prefix and $M_{ri} = 0$ otherwise. Then

$$\begin{aligned} a_{ri} &= v_g^\top \tanh(W_e h_i + W_d d_r) + M_{ri}, \\ \alpha_{ri} &= \mathrm{softmax}_i(a_{ri}), \\ \bar{h}_r &= \sum_i \alpha_{ri} h_i, \\ b_{ri} &= v_p^\top \tanh(U_e h_i + U_g \bar{h}_r) + M_{ri}, \\ P(i_r = i \mid i_{1:r-1}, S_t) &= \mathrm{softmax}_i(b_{ri}). \end{aligned} \tag{30}$$

Equation (30) makes the sequential dependence explicit. The selected prefix updates the decoder state and the next conditional distribution. Masking excludes previously selected cuts, and the glimpse summarizes the candidate pool for the current prefix. The resulting sequence model combines contextual candidate encoding, prefix-aware pointer decoding, and the proposed representation and completion stages into one role-aware selector.

Training keeps stochastic sampling from the policy for exploration. During formal inference, instead of committing to one path at every step, the method maintains $B = 3$ partial sequences. For parent beam $b = (i_1, \ldots, i_{r-1})$ and an unselected extension j, the accumulated score is

$$L_r(b \oplus j) = L_{r-1}(b) + \log P(i_r = j \mid b, S_t), \tag{31}$$

All parent-beam–candidate extensions are compared and the three largest L_r values are retained. After the high-level target length k_t is reached, the best sequence is passed to the completion stage. This beam procedure preserves multiple high-probability partial sequences during decoding, reduces early commitment, and works jointly with the subsequent policy-anchored role-submodular completion to inject variable-role structure into the final selected set.

3.4. Stepwise selection from candidate pool to ordered cuts

At separation callback t, the method executes the following sequence.

Step 1: form the candidate pool. SCIP first generates and validates candidate rows C_t with its own separators. To avoid traversing the complete sparse support of thousands of rows, the method uses SCIP's inexpensive efficacy accessor to retain at most 128 candidates. Ties are broken by original index for deterministic input. Forced cuts remain under solver control and are not placed in the learned ranking.

Step 2: compute and encode the state. The 13 HEM and 10 role-aware features are extracted for every row, with role masses, dominant role, and structural entropy retained as metadata. Equation (28) then performs grouped gated fusion and Eq. (29) produces contextual candidate states. The encoded candidate set is S_t.

Step 3: determine the retained count. The high-level policy outputs a tanh-bounded action in $[-1, 1]$, which is mapped to a retained fraction ρ_t. The fraction is then clipped by the global 0.2 cap, SCIP's round-specific limit, and the implementation cap of 16 cuts, producing k_t. When enough candidates exist, at least two rows are retained to prevent repeated callbacks from degenerating to an empty selection.

Step 4: construct the candidate order. At low-level step r, the pointer policy conditions on S_t and the selected prefix $i_{1:r-1}$ to score all remaining candidates. Selected indices are masked with negative infinity and cannot be repeated. The joint high- and low-level policy is

$$\pi_\theta(k_t, i_{1:k_t} \mid S_t) = \pi_\theta^H(k_t \mid S_t) \prod_{r=1}^{k_t} \pi_\theta^L(i_r \mid S_t, i_{1:r-1}). \tag{32}$$

Equation (32) means that the high level determines the sequence length and the low level then chooses distinct candidates within that length. Each earlier cut changes both the remaining action set and the decoder state used by later probabilities.

Step 5: beam decoding. The low-level policy samples stochastically during training. At test time, beam search of width three is used. At step r, every partial sequence is extended by every unselected candidate; joint log-probabilities are accumulated, and the three highest-scoring items over all parent-beam–next-candidate pairs are retained. After k_t steps, the best sequence is returned. Unlike one-step greedy decoding, the beam retains an initially lower-ranked prefix whose later joint probability can be higher. This fixed-width neural construction follows the broader use of beam decoding in neural combinatorial optimization [41].

Step 6: role-submodular completion. A sequence based only on joint probability can still repeatedly cover one variable role. The first $\lceil 0.25 k_t \rceil$ pointer candidates are retained as policy anchors. The remaining policy order is merged with a heuristic tail sorted by instantaneous structural quality q_i, and at most $2k_t$ rows form expansion pool $\mathcal{R}_t$. For $S \subseteq \mathcal{R}_t$, define

$$\begin{aligned} F_t(S) = &\sum_{i \in S} (0.35 q_i + 0.20 r_i) + 0.25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} \\ &+ \frac{0.20}{|\mathcal{R}_t|} \sum_{u \in \mathcal{R}_t} \max_{i \in S} s_{ui}, \end{aligned} \tag{33}$$

where r_i is the normalized pointer-rank prior, d_g is the demand share of role g in the expansion pool, and s_{ui} is clipped cosine similarity on generic-quality and role-mass descriptors. The score q_i combines efficacy, violation, integral support, recognized-role ratio, dominant-role ratio, and structural entropy. In Eq. (33), the first term retains instantaneous quality and the learned order; the square-root coverage term gives diminishing marginal gain to repeated role mass; and the final term makes the selected cuts represent the main structures in

the pool. Starting from the anchors, the method recomputes the marginal gain of every unselected candidate and adds the largest until k_t cuts have been selected. The nonnegative modular, concave-over-modular, and facility-location terms form a monotone submodular objective, so the residual greedy completion after fixing anchors and the expansion pool inherits the standard $1 - 1/e$ approximation property [42].

Finally, the k_t selected rows are placed at the front of the returned array in the chosen order, while all other candidates remain as an unselected suffix. SCIP adds only the prefix to the current LP. One callback therefore completes six operations: inexpensive truncation, feature extraction, count prediction, sequence decoding, structural completion, and return to SCIP.

Algorithm 1: Cut selection by parallel hierarchical REINFORCE and beam search

Algorithm steps
Input: SCIP-validated rows C_t, high-level policy π_θ^H, low-level pointer policy π_θ^L, beam width $B = 3$, and the current SCIP cut limit.
Output: Ordered cut prefix $S_t^\star = (i_1, \ldots, i_{k_t})$ and the unselected suffix returned to SCIP.
1. Train the policies. Two workers collect rollouts in parallel with terminal reward $R = -$PDI; a central learner applies hierarchical REINFORCE with separate EMA baselines and synchronous Adam updates. Fix the parameters for formal inference.
2. Form the pool. Sort by efficacy, break ties by original index, and retain at most 128 rows.
3. Encode the state. Compute 13 HEM features, ten role-aware features, and role-mass metadata; apply grouped projections, sigmoid-gated fusion, and LSTM encoding.
4. Determine the count. Predict the retained fraction with π_θ^H and obtain k_t after the 0.2 fraction cap, SCIP round limit, and 16-cut implementation cap.
5. Initialize the beam. Set $\mathcal{B}_0$ to the empty sequence with log probability zero.
6. Expand. For $r = 1, \ldots, k_t$, obtain masked conditional probabilities from one glimpse plus pointer attention, extend every prefix in $\mathcal{B}_{r-1}$ by every unselected candidate, and accumulate log probability.
7. Prune. Retain the three highest joint-probability parent–candidate extensions as $\mathcal{B}_r$; after the loop, take the leading sequence in $\mathcal{B}_{k_t}$.
8. Fix anchors. Retain its first $\lceil 0.25 k_t \rceil$ rows as policy anchors A_t.
9. Form the expansion pool. Merge the remaining pointer order with structural-quality order, remove duplicates, and keep at most $2k_t$ rows as $\mathcal{R}_t$.
10. Complete submodularly. Set $S \leftarrow A_t$; while $\lvert S\rvert < k_t$, add the $i \in \mathcal{R}_t \setminus S$ maximizing $F_t(S \cup \{i\}) - F_t(S)$.
11. Return. Return $S_t^\star$ in anchor-then-completion order; leave all other rows in the suffix and outside the current LP.

3.5. Reinforcement-learning training and exact-solver interface

One time-limited SCIP solve forms a rollout, with its terminal PDI used as the cost. The current policy is executed in every eligible cut-selection callback. For replay, the agent records the high-level fraction and low-level sampled sequence at the first valid candidate set of that rollout, and the PDI of the complete solve is returned to both policies. Writing $R = -$PDI as a reward, the low-level estimator is

$$\widehat{\nabla_\theta J} = \frac{1}{B} \sum_{b=1}^{B} (R_b - \bar{R}_{\mathrm{EMA}}) \nabla_\theta \log \pi_\theta(\tau_b), \qquad \bar{R}_{\mathrm{EMA}} \leftarrow 0.9 \bar{R}_{\mathrm{EMA}} + 0.1 \bar{R}. \tag{34}$$

Equation (34) gives the batch policy-gradient update formed from the first eligible candidate-set action record and an exponential-moving-average baseline; B is the rollout batch and τ_b is the recorded cut sequence in rollout b. Equivalently, the code minimizes PDI-cost advantage times log probability. Low-level log probability is averaged over sequence length so that a longer cut sequence does not produce a larger gradient merely because it contains more terms. The high-level policy uses the same PDI cost and an independent EMA baseline and is updated every two epochs; both gradient norms are clipped at 2. Parallel environments collect rollouts, and a central learner updates both policies synchronously after each batch, forming a parallel hierarchical REINFORCE procedure [43, 44].

Training uses five instances and five random seeds. Each seed has two workers, each completing four 30-s/1024-MB rollouts per epoch, for eight rollouts per epoch and 60 epochs. Initial Adam learning rates are 10^{-4} for the low-level policy and 5×10^{-4} for the high-level policy and are multiplied by 0.96 every ten epochs. Training uses stochastic decoding and formal inference uses beam width three. Every configuration–seed combination uses the fixed epoch-60 parameters without benchmark- or test-set checkpoint selection. HEM-13D, feature-only 23D, and the complete 23D method are trained separately so that weights are never mixed across input semantics or completion environments.

RDMCT-HBS only orders and selects valid rows already generated by SCIP, and SPBS submits only solutions checked in the complete model. Neither changes the objective, constraints, integrality, or bound logic. The learned components therefore change the search path and finite-time bound convergence, whereas feasibility and optimality retain the same meaning as in default SCIP.

4. Manufacturing-system experiments and decision analysis

4.1. Experimental design and metrics

The experiments follow the manufacturing-system decision chain and address four questions. First, can configuration inputs be mapped consistently to a complete timeline satisfying dual-source routes, full occupancy, cleaning, and shared-resource constraints while identifying the preferred mode under equal total processing time and different cleaning frequencies? Second, after the throughput structure is determined, does two-stage optimization improve waiting and cadence under the same total budget? Third, do SPBS and structure-aware cut selection improve finite-time plan delivery and primal–dual convergence relative to the direct HEM baseline, respectively? Fourth, can the decision chain continue to return verifiable plans as scale, mixed-flow composition, operating parameters, and the data distribution change? The core evidence comprises cleaning-frequency mode checks, the direct HEM comparison, 960 paired SPBS runs, 1500 equal-budget workflow runs, 300 factorial runs, 80 one-factor sensitivity runs, and 729 distribution-shift runs on 40–64-wafer batches. The main text retains four resource-level Gantt charts for the same 16-wafer batch, grouped by $H = 10$ and $H = 3$; each group shows both pure 4×1 and pure 2×2. They verify process overlap, cleaning, and end-to-end completion time directly. Every incumbent used for a reported conclusion passes both SCIP validation and an independent post-solve check against the same serialized MILP.

OpenAI Codex assisted with research-code debugging and visualization scripts. Numerical results are produced by the

recorded solver runs; event plots are generated programmatically from the validated schedules. The MILP, SCIP version, instances, and random seeds are frozen, and each comparison family uses common time and memory limits. Let $P(t)$ denote the primal bound of the minimization problem at time t, namely the objective value of the best incumbent, and let $D(t)$ denote the dual bound, namely the global lower bound certified by the search tree. Define $\Delta(t) = |P(t) - D(t)|$, $m(t) = \min\{|P(t)|, |D(t)|\}$, and $M(t) = \max\{|P(t)|, |D(t)|\}$. When the two bounds are finite, have the same sign, and differ, the terminal relative primal–dual gap is taken directly from SCIP `getGap()` as

$$G_{\text{end}} = 100\,\frac{\Delta(T)}{m(T)}. \tag{35}$$

SCIP sets $G_{\text{end}} = 0$ when the bounds coincide and returns $+\infty$ when they have opposite signs or valid bounds are not both available. Equation (35) normalizes by the smaller absolute bound and consequently does not cap G_{end} at 100%. A value above 100% indicates that the incumbent and the certified lower bound remain widely separated at the time limit.

SCIP uses a different bounded normalization for the primal–dual integral. Define the instantaneous integral gap as

$$g_{\text{PDI}}(t) = 100\,\frac{\Delta(t)}{M(t)}, \qquad \text{PDI} = \int_0^T g_{\text{PDI}}(t)\,\mathrm{d}t. \tag{36}$$

Equation (36) applies when the bounds are finite, have the same sign, and differ; SCIP sets $g_{\text{PDI}}(t) = 0$ for coincident bounds and $g_{\text{PDI}}(t) = 100$ otherwise. The instantaneous integral gap is therefore bounded to $[0, 100]$; lower PDI means that the bound gap closes earlier. PDI is zero only if the gap is zero for almost the entire timed horizon, whereas $G_{\text{end}} = 0$ only states that optimality is proved at termination. Both quantities are exported directly from SCIP statistics [37, 38].

The manufacturing instance is the inferential unit: random repeats are first aggregated within an instance, after which paired instance differences use nonparametric 95% bootstrap intervals, Wilcoxon signed-rank tests, and Holm correction within each prespecified analysis family. Time-limited runs remain in the summaries so that random seeds are not treated as independent manufacturing evidence. Solver environments, budgets, tolerances, instance lists, and model parameters are recorded separately for each comparison family; random repeats are aggregated by manufacturing instance.

The numerical values in Table 3 define the computational configuration. The product-return comparison uses equal cumulative nominal processing time: 180 s for one 4×1 exposure and 90+90 s for two 2×2 exposures. Recipe qualification is a prescribed input condition for this scheduling comparison.

Table 3 lists the frozen configuration used to produce the representative 16-wafer Gantt charts. Two rotating chambers share the VTR, while product and dummy wafers compete for robots, load locks, and chamber spaces along their respective routes; cleaning counts, rotation, and transfer times are represented in the same MILP. Each displayed solution passes SCIP `checkSol` and an independent post-solve check before export, so the charts provide both a structural comparison and auditable feasibility evidence.

4.2. *Process-mode selection driven by cleaning position and lot size*

Steady-state analyses of multi-space CTs usually measure cycle time or throughput [4]. For finite batches, we define $C_{\max}$ as the time when the last product wafer returns to an LP and report equivalent batch output as WPH $= 3600|\mathcal{W}|/C_{\max}$. Pure 4×1 uses 180 s per wafer, pure 2×2 uses 90+90 s, and a 1:1 mixture assigns each product its mode-specific recipe. The cleaning interval H counts full-occupancy cycles per chamber: a 4×1 batch counts once and each 2×2 exposure counts separately. Processing segments follow cycle limits and complete-chain boundaries, including 500-s cleaning and associated dummy loading, unloading, rotation, and transport.

Table 4 compares cleaning endpoints; Table 5 gives the full $H = 3$–10 grid. The $H = 4$–9 experiment contains 36 configurations with training and solver seeds fixed at 1, a 14400-s main-solve limit, and stage 2 disabled. All 36 schedules pass independent validation; 35 are proven optimal. For 16 wafers at $H = 5$, pure 2×2 reaches a feasible 907 s within that budget. A separate 28800-s solve of the same frozen instance, initialized from this schedule, improves the upper bound to 898 s, with lower bound 694 s and terminal gap 29.39%. The improved schedule passes main-model and 4-s exchange checks. This certification updates the schedule bound and is excluded from the fixed-budget algorithm statistics. With cleaning inactive, the 16-wafer pure 2×2 models at $H = 6$–10 are identical and have optimum 898 s.

For 8 wafers, the three makespans remain 610, 578, and 654 s throughout $H = 3$–10. Pure 2×2 is 5.25% shorter than pure 4×1. At $H = 3$, pure 2×2 and mixed operation generate two and one terminal cleans, respectively; no cleaning occurs at $H \geq 4$. No internal cleaning is required at this lot size, and the ranking remains unchanged across the tested intervals.

For 16 wafers, the preferred mode changes with the interval. At $H = 3$, pure 2×2 and mixed operation each contain two internal cleans, and pure 4×1 is preferred at 944 s. At $H = 4$, pure 2×2 still requires two internal cleans, while mixed operation has only two terminal cleans and improves on pure 4×1 at 939 s. At $H = 5$, pure 2×2 has only two terminal cleans and its feasible 898-s schedule is shorter than the proven optima of both competitors. Thus the preference between the pure modes switches from 4×1 at $H = 4$ to 2×2 at $H = 5$. At $H = 6$–10, all modes avoid cleaning; the pure 2×2 optimum of 898 s is 4.87% and 4.37% shorter than pure 4×1 and mixed operation, respectively.

Complete-chain segmentation and cleaning position jointly explain this change. Each chamber in the 16-wafer pure 2×2 configuration processes four product PWs. The layout splits these into 2+2 PWs at $H = 3$, 3+1 at $H = 4$, and one complete chain at $H \geq 5$. Although $H = 3$ and 4 have equal internal-cleaning counts, their supply and return orders differ; the latter layout yields 1470 s. Mode selection therefore depends on lot size, recipe composition, and admissible complete-chain length. The experiments reconstruct preset complete-chain layouts for each H, so the comparison reflects the joint effect of the cleaning limit and chain layout.

Table 3: Frozen configuration for the cleaning-frequency mode experiment and representative Gantt charts under equal total nominal processing time.

Group	Parameter and value	Group	Parameter and value
Rotating chambers	CH2 and CH3; two shared rotating PMs with four spaces each	Process time	180 s for 4×1; 90+90 s for 2×2
Cleaning	At most $H = 3$–10 effective process cycles between adjacent cleanings; pure-dummy cleaning 500 s	Chamber motion	180° rotation 2 s; inter-action gap 1 s
ATR	One dual-hand robot, capacity 2; load/unload 4 s, complete loaded transfer 11 s	VTR	One four-hand robot, capacity 4; loaded transfer 4 s, empty reposition 4 s
ATR details	LL–LP empty return 6 s; PW-pair exchange at AL 8 s	AL	Capacity 1; minimum alignment occupancy 20 s per wafer
Load locks	LLupper pump/reset 30 s per slot; LLlower vent/reset 25 s	LL capacity	Two independent slots in each of upper and lower LLs
Material source	LP1–LP4, capacity 25 wafers each	Dummy wafers	Eight reusable wafers, four per chamber; storage capacity 10
Batch and model	Product batch size 4; 13 modeled process steps	Chamber bound	At most 15 4×1 batch slots per PM
Model bounds	Big-M = 10000; module and robot residency bounds 10000 s	Waiting option	Avoidable-wait hard cap 0, meaning that this optional cap is disabled
Objective	`compact_resource_idle`; process mode `auto`	Policy	Frozen training seed 1, epoch 60, 23 features
Decoder and cuts	Beam width 3; cut pool 128, at most 16 cuts selected per round	Solver	SCIP seed 1; Available, validated initial schedules
Solve and selection	Product-return objective; optimum certification and representative-schedule waiting refinement recorded separately	Export check	SCIP `checkSol` and independent validation for every solution

Table 4: Cleaning-endpoint comparison under equal cumulative nominal processing time. Cleaning counts sum internal and terminal cleans across both chambers. † marks a validated feasible makespan without an optimality proof; the corresponding WPH uses that makespan.

Lot (wafers)	H	Mode	Total cleans	$C_{\max}$ (s)	WPH (wafers/h)
8	10	Pure 4×1	0	610	47.21
8	10	Pure 2×2	0	**578**	**49.83**
8	10	4:4 mixed	0	654	44.04
8	3	Pure 4×1	0	610	47.21
8	3	Pure 2×2	2	**578**	**49.83**
8	3	4:4 mixed	1	654	44.04
16	10	Pure 4×1	0	944	61.02
16	10	Pure 2×2	0	**898**	**64.14**
16	10	8:8 mixed	0	939	61.34
16	3	Pure 4×1	0	**944**	**61.02**
16	3	Pure 2×2	4	1402	41.08
16	3	8:8 mixed	2	1530	37.65
24	10	Pure 4×1	0	1268	68.14
24	10	Pure 2×2	0	**1237**†	**69.85**
24	10	12:12 mixed	0	1250	69.12
24	3	Pure 4×1	2	**1268**	**68.14**
24	3	Pure 2×2	6	2220†	38.92
24	3	12:12 mixed	4	2324†	37.18

The 24-wafer extension covers all three modes at $H = 3$–6 with 7200 s per configuration. Pure 4×1 has optimum 1268 s at each interval. At $H = 4$, pure 2×2 and mixed-mode lower bounds are 1342 and 1774 s; at $H = 5$, the pure 2×2 lower bound is 1440 s and the mixed optimum is 1796 s. These bounds establish pure 4×1 as preferred at both intervals. At $H = 6$, the mixed-mode upper bound of 1263 s is below the pure 4×1 optimum and pure 2×2 lower bound of 1440 s, establishing mixed-mode dominance; its terminal gap is 10.40%. At $H = 3$, a 28800-s certification run for pure 2×2 obtains a validated 2220-s schedule, lower bound 2082 s, and gap 6.63%. This lower bound and the mixed-mode lower bound of 2302 s exceed the pure 4×1 optimum. The certification is used for mode ordering, not fixed-budget algorithm comparison. Hence pure 4×1 is preferred for 24 wafers at $H = 3$–5 and mixed operation at $H = 6$. The difference from the 16-wafer $H = 5$ result demonstrates the dependence on lot size and complete-chain layout.

All 12 configurations in the 24-wafer $H = 7$–10 extension yield validated schedules. An independent certification run for mixed operation at $H = 10$ proves a makespan of 1250 s within the 28800-s budget. The mixed-mode MILP files at $H = 7$–10 have identical SHA-256 hashes, so this certificate applies to all four intervals. Pure 2×2 yields a feasible 1237-s schedule at $H = 8$–10. Transferring its production-event timing to the $H = 7$ model and scheduling two terminal cleanings also gives a 1237-s schedule that passes validation against the original model. The valid pure 2×2 intervals are $[712, 1237]$ s at $H = 7$ and $[842, 1237]$ s at $H = 8$–10. These upper bounds are below the mixed-mode optimum of 1250 s and pure 4×1 optimum of 1268 s. Thus, pure 2×2 is preferred throughout $H = 7$–10, with the constructed schedules reducing makespan by 1.04% and 2.44%, respectively. Certification and feasibility-transfer costs are accounted for separately from the fixed-budget algorithm comparison.

The four resource Gantt charts use 16 wafers and compare the pure modes at $H = 10$ and 3. To study waiting at minimum makespan, the $H = 10$ pure 2×2 schedule is further optimized for total route wait under $C_{\max} \leq 898 + 10^{-6}$ s, with resource order allowed to change. The minimum route wait is 492 s,

Table 5: Makespans, cleaning counts, and preferred modes under the prescribed PW layouts for 8-, 16-, and 24-wafer lots at $H = 3$–10. Times are in seconds.

Lot (wafers)	H	$C_{\max}$ (s)			$\Delta C_{\max}$ (s)	Total cleans			Preferred mode
		Pure 4×1	Pure 2×2	1:1 mixed		Pure 4×1	Pure 2×2	1:1 mixed	
8	3	610	578	654	-32	0	2	1	Pure 2×2
8	4	610	578	654	-32	0	0	0	Pure 2×2
8	5	610	578	654	-32	0	0	0	Pure 2×2
8	6	610	578	654	-32	0	0	0	Pure 2×2
8	7	610	578	654	-32	0	0	0	Pure 2×2
8	8	610	578	654	-32	0	0	0	Pure 2×2
8	9	610	578	654	-32	0	0	0	Pure 2×2
8	10	610	578	654	-32	0	0	0	Pure 2×2
16	3	944	1402	1530	458	0	4	2	Pure 4×1
16	4	944	1470	939	526	0	2	2	1:1 mixed
16	5	944	898†	939	-46	0	2	0	Pure 2×2
16	6	944	898	939	-46	0	0	0	Pure 2×2
16	7	944	898	939	-46	0	0	0	Pure 2×2
16	8	944	898	939	-46	0	0	0	Pure 2×2
16	9	944	898	939	-46	0	0	0	Pure 2×2
16	10	944	898	939	-46	0	0	0	Pure 2×2
24	3	1268	2220†	2324†	952	2	6	4	Pure 4×1
24	4	1268	2928†	1827†	1660	0	4	2	Pure 4×1
24	5	1268	1769†	1796	501	0	2	2	Pure 4×1
24	6	1268	1880†	1263†	612	0	2	1	1:1 mixed
24	7	1268	1237†	1250	-31	0	2	0	Pure 2×2
24	8	1268	1237†	1250	-31	0	0	0	Pure 2×2
24	9	1268	1237†	1250	-31	0	0	0	Pure 2×2
24	10	1268	1237†	1250	-31	0	0	0	Pure 2×2

Cleaning counts sum internal and terminal cleans across both chambers. † denotes a feasible upper bound; other times are proven optima. $\Delta C_{\max}$ subtracts the listed pure 4×1 time from the pure 2×2 time. A preferred mode is established by comparing optima or separating its feasible upper bound from competing lower bounds. The 24-wafer $H = 7$ pure 2×2 entry uses a transferred and revalidated 1237-s schedule. Mixed-mode entries at $H = 7$–10 share the 1250-s optimality certificate obtained within a 28800-s budget; their model hashes are identical. The 16-wafer $H = 5$ and 24-wafer $H = 3$ pure 2×2 entries also use 28800-s certification runs; other 24-wafer results use 7200-s experiments. Mixed-mode $H = 3, 5$ times use high-precision timing reoptimization of the same discrete schedule. All configurations allow pre-exchange waiting and split exchanges.

accumulated over six consecutive-operation intervals for each product wafer: 448 s from AL pair coordination, 36 s before post-processing unloading, and 8 s before return from the lower load lock. This route measure is distinct from the interval between consecutive in-chamber exposures. Each time axis ends at the last product return, $C_{\max}$. Every pure 4×1 process lasts 180 s and every pure 2×2 exposure lasts 90 s.

Chamber lanes distinguish gray exchange actions from pale-pink waiting. A combined exchange by the four-finger VTR takes 4 s. A split exchange includes 4-s unloading, at least 4 s of empty repositioning, and 4-s loading; other material actions may be coordinated between unloading and loading. In Fig. 6, the first CH2 inter-exposure interval is 63 s: 59 s waiting followed by a 4-s combined exchange. The next product PW, W5/W6, enters the upper load lock at 299 s and completes vacuum-side preparation at 329 s. CH2 finishes processing at 270 s, waits until 329 s, and exchanges at 329–333 s. Most adjacent cycles in the $H = 3$ short-chain schedule use continuous 4-s combined exchanges; CH2 cycles five and six are separated by waits at 1106–1108 s and 1112–1116 s, unloading at 1108–1112 s, and loading at 1116–1120 s. The final exposure starts immediately at 1120 s and ends at 1210 s. Every exposure remains 90 s. Constraining all eight inter-exposure intervals in the same $H = 10$ model to exactly 4 s yields an independently validated optimum of 960 s, 62 s (6.90%) above 898 s. Continuous exchanges compress local intervals but restrict supply and return order on shared resources; allowing local waiting shortens this finite batch's makespan.

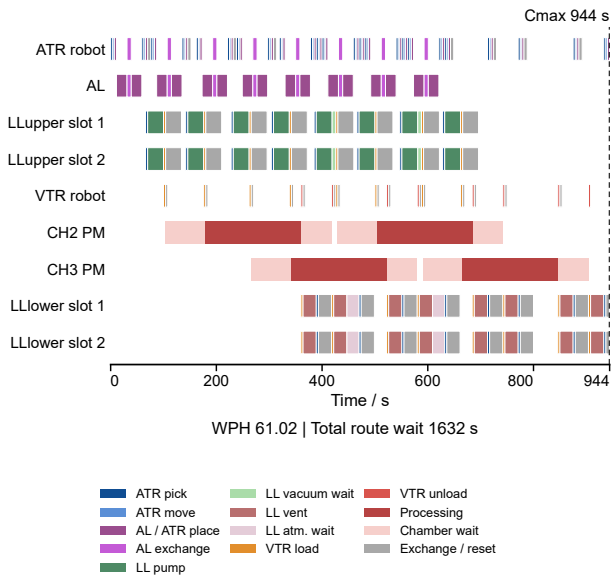


Figure 5: 16-wafer pure 4×1 schedule at $H = 10$. No cleaning occurs; $C_{\max} = 944$ s, WPH is 61.02 wafers/h, and total route wait is 1632 s.

4.3. *Equal-budget refinement of waiting and cadence*

Waiting-time constraints can propagate to manufacturing-system output and control performance [26]. For the same 600-s total budget, the complete two-stage workflow assigns 540 s to the main solve and up to 60 s to refinement, whereas the one-stage control assigns all 600 s to the main solve. Figure 9 shows

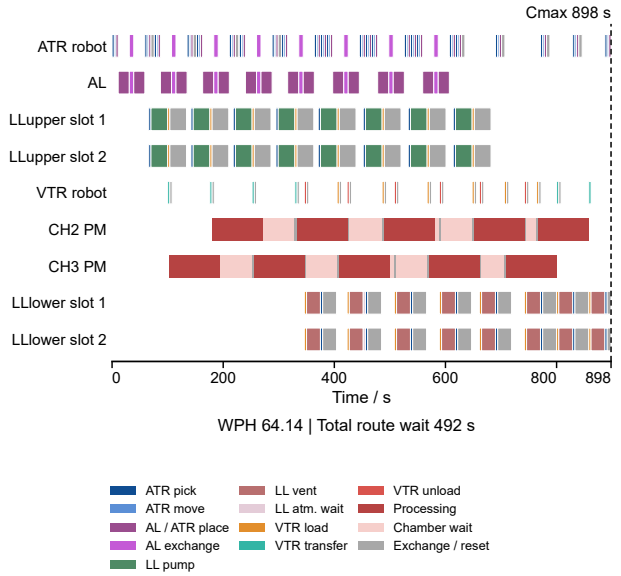


Figure 6: 16-wafer pure 2×2 schedule at $H = 10$, without cleaning: $C_{\max} = 898$ s, WPH = 64.14 wafers/h. Refinement minimizes total route wait to 492 s at this makespan. Gray denotes 4-s chamber exchanges; pale pink denotes waiting.

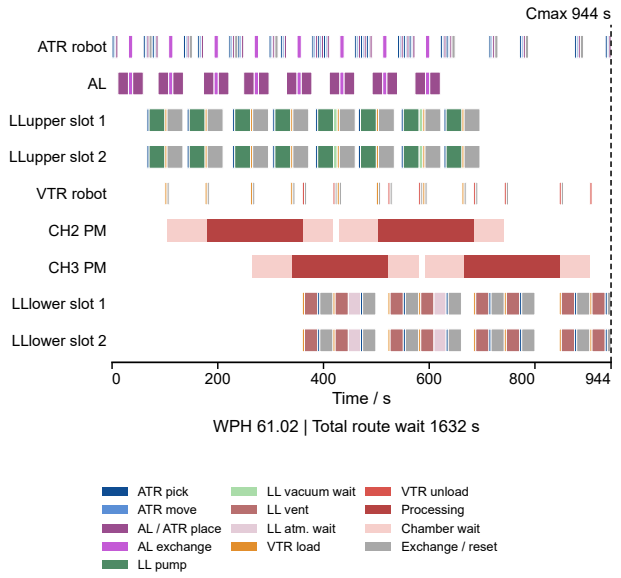


Figure 7: 16-wafer pure 4×1 schedule at $H = 3$: $C_{\max} = 944$ s, WPH is 61.02 wafers/h, and total route wait is 1632 s.

that the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. All four paired instance differences remain significant after Holm correction, demonstrating that the two-stage workflow markedly improves waiting and cadence quality under the same total budget.

The quadratic-objective control and the complete linear refinement both use the 540+up-to-60-s split. Linear refinement yields a cadence deviation of 22.04, 63.38% below the quadratic profile's 60.18, and the matched difference remains significant after Holm correction ($p = 0.0377$); total route wait, maximum wait, and cadence coefficient of variation are similar overall. The linear objective therefore reduces the measured cadence deviation under the same budget.

Table 6 isolates the second stage and refinement objective, complementing the initialization ablation in Table 7.

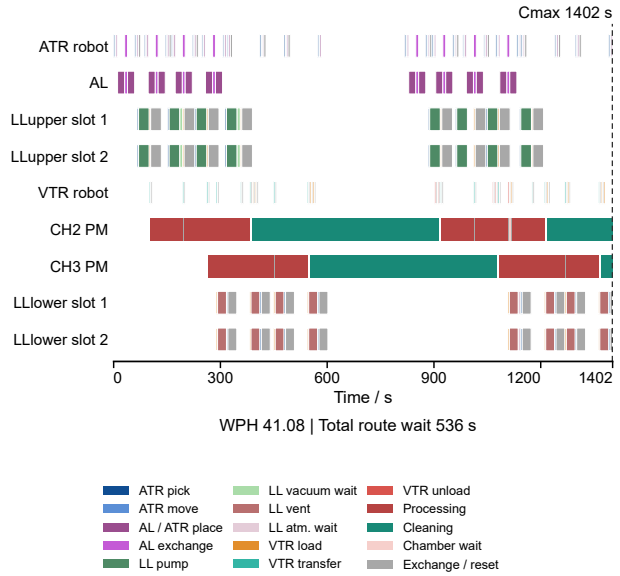


Figure 8: 16-wafer pure 2×2 schedule at $H = 3$: two internal and two terminal cleans, $C_{\max} = 1402$ s, WPH = 41.08 wafers/h, and total route wait = 536 s. The horizon ends at product return; exchanges and waiting are shown separately.

4.4. SPBS and structure-aware cut selection

Table 7 summarizes the paired SPBS study with 48 instances and ten paired seeds per instance. The timed comparison measures main-solver performance after offline SPBS construction. Relative to the no-start condition, precomputed SPBS increases incumbent availability from 51.67% to 100%, a gain of 48.33 percentage points (95% interval [34.58, 62.29]). Mean PDI falls from 27473.05 to 20278.78, a decrease of 7194.27 or 26.19% (95% interval [5145.78, 9317.61]). Both paired differences remain significant after Holm correction ($p = 2.55 \times 10^{-5}$ and $p = 3.46 \times 10^{-8}$). Precomputed SPBS therefore improves feasible-schedule availability and primal–dual convergence during the subsequent solve.

HEM-13D is the most direct learning baseline for the structure-aware cut selector. Both methods share SPBS, training instances, random seeds, candidate limits, hidden dimensions, and a 600-s solve budget, with 500 runs per method. Table 8 reports this prespecified direct comparison. Relative to HEM-13D, RDMCT-HBS changes mean PDI from 16823.18 to 16585.70, mean solve time from 219.65 to 218.32 s, mean terminal gap from 127.24% to 126.01%, and proven-optimal rate from 62.4% to 63.2%. All 500 complete-method runs produce

Table 6: Ablation of second-stage refinement and objective form. Each configuration contains 20 instances and 500 runs under a 600-s total budget. Entries are instance-aggregated means; lower is better.

Configuration	Total wait (s)	Max. wait (s)	Cadence CV	Deviation (s)
Full linear refinement	2596.01	296.33	0.130	22.04
Quadratic refinement	2469.47	293.27	0.134	60.18
Without stage two	5398.60	952.57	0.216	370.41

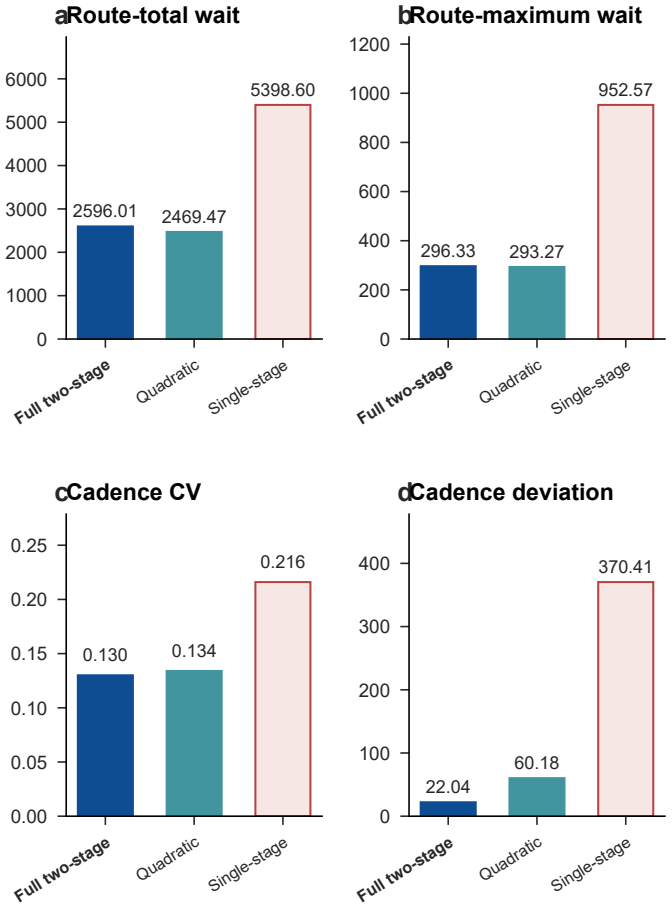


Figure 9: Final-schedule quality under the same 600-s total budget. a–d, total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation; 1500 runs in total, with observed means labeled above the bars.

Table 7: Effect of precomputed SPBS on time-limited MILP search; construction time is excluded. Each condition contains 48 instances and ten paired seeds per instance, totaling 480 runs.

Metric	No start	SPBS	Observed change
Incumbent rate (%)	51.67	100.00	+48.33 points
Mean PDI	27473.05	20278.78	-26.19%

feasible schedules. After aggregating repeats over 20 manufacturing instances, all four paired mean-difference 95% bootstrap intervals include zero and all Holm-adjusted p-values are 1.0. The observed means are close, and the paired tests do not detect significant differences under the common SPBS condition.

The comparison evaluates role-aware representation, sequence decoding, and structural completion jointly under common initialization and solver budgets. It finds no significant performance difference from HEM-13D on the tested instances.

Table 8: Direct comparison of RDMCT-HBS and HEM-13D under a common SPBS and budget.

Metric	HEM-13D	RDMCT-HBS	Observed change
Mean PDI	16823.18	16585.70	-1.41%
Mean solution time (s)	219.65	218.32	-0.61%
Mean terminal gap (%)	127.24	126.01	-0.97%
Proven-optimal rate (%)	62.4	63.2	+0.80 points

4.5. Scale, recipe composition, and computational difficulty

The factorial design contains $4 \times 5 \times 3 = 60$ wafer-count–recipe-composition–process-scale cells with five solver seeds per cell, for 300 runs. Figure 10 shows a sharp transition. All 75 runs at both 8 and 16 wafers prove optimality, with mean PDI values of 28.13 and 1876.54 and mean solution times of 0.34 and 33.86 s. At 24 and 32 wafers, the proven-optimal rates fall to 36% and 20%, mean PDI rises to 31155.94 and 32959.89, and mean solution time reaches 408.55 and 433.20 s.

The prespecified PDI factorial model has $R^2 = 0.998$. Under the 1:1 mixed recipe, the additional interaction coefficients at 24 and 32 wafers are 37729.48 and 40788.63, while the 95% intervals for the process-scale main effect and all prespecified process-scale interactions include zero. Over the tested 0.9–1.1 process-scale range, computational difficulty is driven primarily by the interaction between batch size and mixed-flow composition, with a comparatively small process-scale effect.

4.6. Operation under parameter perturbations

Prior CT studies show that cleaning actions and their triggering conditions alter robot-task sequences, cycle time, and wafer delay [21, 23]. The one-factor sensitivity study therefore covers eight prescribed perturbations of transport, processing, cleaning interval, and vacuum-domain dummy inventory, for 80 runs. Every run returns a solver-checked feasible schedule and 20 runs prove optimality; scenario mean PDI ranges from 28899.41 to 42478.32. A longer cleaning interval gives the lowest mean PDI and slower transport the highest, describing the variation in search performance across the tested parameter settings.

4.7. Feasible-schedule delivery under distribution shift

The distribution-shift suite enlarges batches to 40–64 wafers and changes mixed-flow composition, producing 729 runs across all compared methods over nine shifted manufacturing scenarios, including 135 runs of the complete method. This experiment evaluates schedule availability and search performance under changes in scale and composition. All 729 runs return an incumbent that passes SCIP and independent post-solve

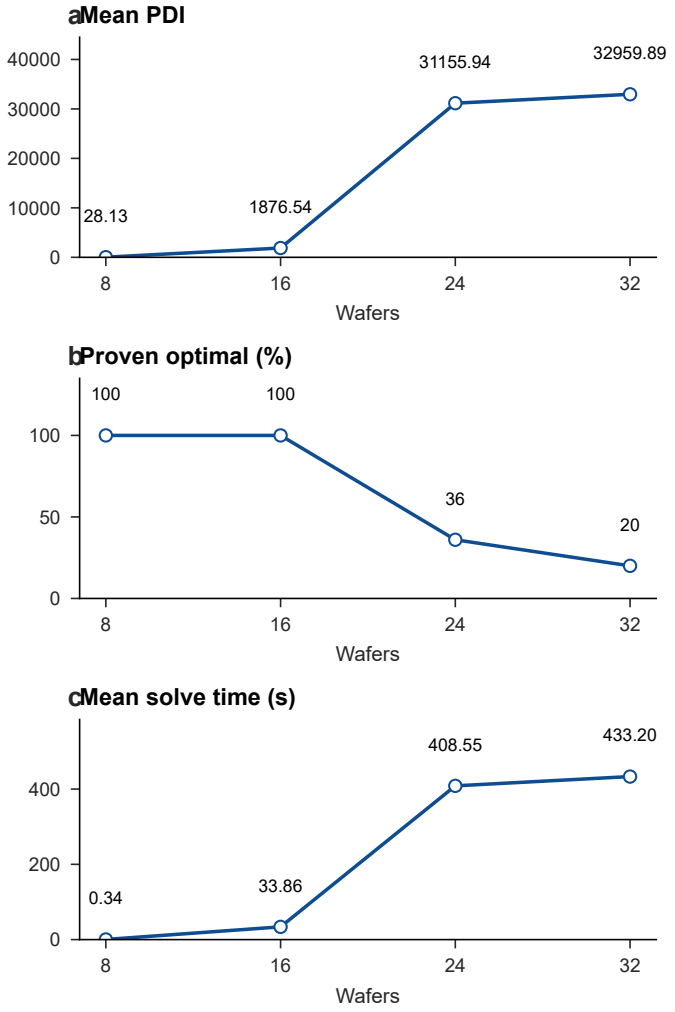


Figure 10: Scale transition in the 300-run factorial experiment. a, mean PDI; b, proven-optimal rate; c, mean solution time. Each wafer count contains 75 runs and observed values are labeled directly.

checks. The shared SPBS–MILP–SCIP chain therefore continues to output a validated resource-action schedule over the tested changes in scale and recipe composition. No run proves optimality. The complete method has mean PDI 73919.15, compared with 73670.99 for default SCIP and 70915.54 for adaptive cut scoring. Feasible-schedule availability is 100% across this suite, while adaptive cut scoring has the lowest observed mean PDI.

4.8. Manufacturing-system integration and operational implications

The framework organizes equipment-cell decisions into configuration input, schedule generation, and event output. MES and process management supply batches, sources, and recipes; equipment configuration supplies capacities, dummy inventory, and cleaning rules. The decision layer coordinates dual-source supply, full-occupancy processing, cleaning, and return and reports timed resource events, makespan, and solver status. The proposed controller interface defines mappings from these records to tool actions and feedback fields for actual completion times, resource states, and frozen actions.

The cleaning grid provides quantitative mode-selection guidance for a given lot. Pure 2×2 is preferred for 8 wafers throughout $H = 3$–10. For 16 wafers, pure 4×1, 1:1 mixed operation, and pure 2×2 are preferred at $H = 3$, $H = 4$, and $H = 5$–10, respectively. At $H = 4$, mixed operation has only terminal cleaning and yields the shortest makespan of 939 s; at $H \geq 5$, pure 2×2 can use one complete processing chain per chamber. These comparisons connect batch mode configuration to cleaning organization.

Different components support schedule delivery and timing quality. SPBS achieves 100% feasible-schedule availability in its experiment, while the equal-budget two-stage workflow significantly reduces waiting and cadence variation. The two learning-based cut selectors have similar average performance with common SPBS, with no statistically significant paired differences. Across all methods, the scale extension delivers feasible schedules in 729 runs with 40–64 wafers, including 135 runs of the complete method. Together, these results connect configuration, material coordination, timing quality, and solver status within an equipment-cell decision process.

5. Conclusions

This study develops an operational decision framework for dual-source mixed-flow PECVD rotating-chamber equipment. Product and dummy PWs connect atmospheric-side and vacuum-side logistics. The two-stage MILP jointly represents full-occupancy processing, dummy reuse, periodic cleaning, residency, and shared-resource order and generates complete finite-batch event schedules.

Experiments on three preset modes with prescribed PW layouts and equal cumulative nominal processing time show that lot size and cleaning interval change mode priority through complete-chain segmentation and cleaning position. Pure 2×2 is preferred for 8 wafers throughout $H = 3$–10. For 16 wafers, pure 4×1, 1:1 mixed operation, and pure 2×2 are preferred at $H = 3$, 4, and 5–10, respectively. For 24 wafers, the corresponding intervals are $H = 3$–5, 6, and 7–10. At 16 wafers and $H = 6$–10, the pure 2×2 optimum is 898 s, 4.87% below pure 4×1. At 24 wafers and $H = 7$–10, its feasible upper bound of 1237 s is below both competing optima. Mode selection therefore requires comparison at the relevant lot size and cleaning layout.

Under the same 600-s total budget, the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. With precomputed initial schedules, SPBS increases feasible-schedule availability during the timed main solve from 51.67% to 100% and reduces mean PDI by 26.19%. With common SPBS, RDMCT-HBS and HEM-13D have similar means on all four measures, and paired differences are not significant after multiplicity correction. Across all methods, the 729 distribution-shift runs with 40–64 wafers deliver feasible schedules, including 135 runs of the complete method.

By scheduling dummies as a material flow requiring closed-loop supply, the framework provides a quantitative basis for mode selection, cleaning organization, and equipment-resource coordination. Future work will address rolling scheduling under stochastic arrivals and equipment disturbances and validate controller adaptation and event-feedback integration.

CRediT authorship contribution statement

Lincong Wei was responsible for Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, and Writing – review and editing. Lizhen Shao was responsible for Conceptualization, Methodology, Formal analysis, Validation, Writing – review and editing, and Project administration. Wei Wang was responsible for Formal analysis, Investigation, Writing – review and editing, and Supervision. All authors reviewed and approved the final manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The data, mathematical instances, trained models, and source code supporting the findings are available from the corresponding author on reasonable request.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI ChatGPT and Codex to assist with language editing, Chinese–English translation, LaTeX consistency checks, and figure-production code. Research-code assistance is described with the computational methods. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] Applied Materials, n.d. Producer XP Precision CVD. https://www.appliedmaterials.com/us/en/product-library/producer-xp-precision-cvd.html (accessed 7 September 2026).

[2] Pan, C.R., Zhou, M.C., Qiao, Y., Wu, N.Q., 2018. Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges. IEEE Transactions on Automation Science and Engineering 15, 2, 586–601. doi:10.1109/TASE.2016.2642997.

[3] Gu, L., Wu, N., Qiao, Y., Zhang, S., Li, T., 2024. Scheduling cluster tools with multi-space process modules and a multi-finger-arm robot in wafer fabrication subject to wafer residency time constraints. Applied Sciences 14, 9490. doi:10.3390/app14209490.

[4] Gu, L., Wu, N., Qiao, Y., Zhang, S., 2024. Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules, in: 2024 International Conference on Networking, Sensing and Control (ICNSC), IEEE. doi:10.1109/ICNSC62968.2024.10760109.

[5] Li, Y., Liu, Q., Li, X., Gao, L., 2025. Manufacturing resource-based self-organizing scheduling using multi-agent system and deep reinforcement learning. Journal of Manufacturing Systems 79, 179–198. doi:10.1016/j.jmsy.2025.01.004.

[6] Zhu, T., Liu, X., Yu, Y., Fu, L., 2025. Matrix manufacturing system layout and scheduling via graph neural network and multi-action deep reinforcement learning. Journal of Manufacturing Systems 82, 239–253. doi:10.1016/j.jmsy.2025.06.005.

[7] Bhatta, K., Chang, Q., 2025. Train small, deploy large: Scaling multi-agent reinforcement learning for multi-stage manufacturing lines. Journal of Manufacturing Systems 81, 155–168. doi:10.1016/j.jmsy.2025.04.017.

[8] Chen, D., Zhang, J., Wu, L., Zhang, P., Wang, M., 2024. Industrial data space application framework for semiconductor wafer manufacturing system scheduling. Journal of Manufacturing Systems 77, 464–482. doi:10.1016/j.jmsy.2024.09.013.

[9] Chen, D., Zhang, J., Wu, L., Zhang, P., Lv, Y., Wang, J., Wang, H., 2025. Optimisation method for semiconductor wafer manufacturing system scheduling: Reinforcement learning with decision graph guiding. Journal of Manufacturing Systems 82, 1158–1170. doi:10.1016/j.jmsy.2025.08.004.

[10] Kim, M.G., Kim, H., Barde, S., Lee, C.H., 2025. Balancing yield and makespan in wafer fabrication: A two-stage data-driven scheduling approach. Journal of Manufacturing Systems 82, 874–904. doi:10.1016/j.jmsy.2025.07.009.

[11] Lee, S.W., Lee, D.H., Kim, M.C., Lee, S.Y., Kim, H.J., 2026. A decomposition-based optimization method for parallel machine scheduling under quality uncertainty in wafer manufacturing. Journal of Manufacturing Systems 85, 386–401. doi:10.1016/j.jmsy.2026.02.002.

[12] Perkinson, T.L., McLarty, P.K., Gyurcsik, R.S., Cavin, R.K., 1994. Single-wafer cluster tool performance: An analysis of throughput. IEEE Transactions on Semiconductor Manufacturing 7, 369–373. doi:10.1109/66.311340.

[13] Rostami, S., Hamidzadeh, B., 2002. Optimal scheduling techniques for cluster tools with process-module and transport-module residency constraints. IEEE Transactions on Semiconductor Manufacturing 15, 341–349. doi:10.1109/TSM.2002.801379.

[14] Wu, N.Q., Zhou, M.C., 2010. A closed-form solution for schedulability and optimal scheduling of dual-arm cluster tools with wafer residency time constraint based on steady schedule analysis. IEEE Transactions on Automation Science and Engineering 7, 303–315. doi:10.1109/TASE.2008.2008633.

[15] Qiao, Y., Wu, N.Q., Zhou, M.C., 2012. Petri net modeling and wafer sojourn time analysis of single-arm cluster tools with residency time constraints and activity time variation. IEEE Transactions on Semiconductor Manufacturing 25, 432–446. doi:10.1109/TSM.2012.2199338.

[16] Qiao, Y., Wu, N., Zhou, M.C., 2014. Scheduling of dual-arm cluster tools with wafer revisiting and residency time constraints. IEEE Transactions on Industrial Informatics 10, 286–300. doi:10.1109/TII.2013.2272702.

[17] Wang, J., Hu, H., Pan, C., Zhou, Y., Li, L., 2020. Scheduling dual-arm cluster tools with multiple wafer types and residency time constraints. IEEE/CAA Journal of Automatica Sinica 7, 776–789. doi:10.1109/JAS.2020.1003150.

[18] Wang, J., Liu, C., Zhou, M.C., Abusorrah, A., 2025. Virtual cell-based scheduling approach to single-robotic-arm cluster tools subject to wafer residency time constraints. IEEE Transactions on Automation Science and Engineering 22, 240–251. doi:10.1109/TASE.2024.3349380.

[19] Yang, F., Li, C., Zhen, L., Zhang, C., Wu, N., 2025. Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. Expert Systems with Applications 288, 128209. doi:10.1016/j.eswa.2025.128209.

[20] Kim, H., Kim, H.J., Lee, J.H., Lee, T.E., 2013. Scheduling dual-armed cluster tools with cleaning processes. International Journal of Production Research 51, 3671–3687. doi:10.1080/00207543.2012.758392.

[21] Yu, T.S., Lee, T.E., 2020. Wafer delay analysis and control of dual-armed cluster tools with chamber cleaning operations. International Journal of Production Research 58, 434–447. doi:10.1080/00207543.2019.1593547.

[22] Qiao, Y., Lu, Y., Li, J., Zhang, S., Wu, N., Liu, B., 2022. An efficient binary integer programming model for residency time-constrained cluster

tools with chamber cleaning requirements. IEEE Transactions on Automation Science and Engineering 19, 1757–1771. doi:10.1109/TASE.2021.3122576.

[23] Li, C., Yang, F., Zhen, L., 2022. Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. International Journal of Production Research 60, 3555–3568. doi:10.1080/00207543.2021.1926568.

[24] Zhu, Q., Yuan, J., Wang, G., Hou, Y., 2025. Scheduling a single-arm two-cluster tool with a process module failure subject to wafer residency time constraints. IEEE Transactions on Automation Science and Engineering 22, 667–679. doi:10.1109/TASE.2023.3275703.

[25] Kim, D., Kim, H.J., 2025. Noncyclic scheduling of cluster tools using deep reinforcement learning. IEEE Transactions on Automation Science and Engineering 22, 24009–24023. doi:10.1109/TASE.2025.3632534.

[26] Lee, J.H., Zhao, C., Li, J., Papadopoulos, C.T., 2018. Analysis, design, and control of Bernoulli production lines with waiting time constraints. Journal of Manufacturing Systems 46, 208–220. doi:10.1016/j.jmsy.2018.01.001.

[27] Kim, H.J., Lee, J.H., 2025. Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search. IEEE Transactions on Automation Science and Engineering 22, 3783–3796. doi:10.1109/TASE.2024.3399818.

[28] Hong, T.Y., Hsu, C.Y., Chen, C.L., 2026. Simulation-based deep reinforcement learning for process time optimization in semiconductor cluster tool and an empirical study. Computers & Industrial Engineering 211, 111698. doi:10.1016/j.cie.2025.111698.

[29] Tang, Y., Agrawal, S., Faenza, Y., 2020. Reinforcement learning for integer programming: Learning to cut, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 9367–9376. URL: https://proceedings.mlr.press/v119/tang20a.html.

[30] Huang, Z., Wang, K., Liu, F., Zhen, H.L., Zhang, W., Yuan, M., Hao, J., Yu, Y., Wang, J., 2022. Learning to select cuts for efficient mixed-integer programming. Pattern Recognition 123, 108353. doi:10.1016/j.patcog.2021.108353.

[31] Paulus, M.B., Zarpellon, G., Krause, A., Charlin, L., Maddison, C., 2022. Learning to cut by looking ahead: Cutting plane selection via imitation learning, in: Proceedings of the 39th International Conference on Machine Learning, PMLR. pp. 17584–17600. URL: https://proceedings.mlr.press/v162/paulus22a.html.

[32] Wang, Z., Li, X., Wang, J., Kuang, Y., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2023. Learning cut selection for mixed-integer linear programming via hierarchical sequence model, in: Proceedings of the Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=Zob4P9bRNcK.

[33] Bloom, H., 1990. Software and Computer Integrated Manufacturing. NISTIR 4468, National Institute of Standards and Technology. doi:10.6028/NIST.IR.4468.

[34] OPC Foundation, n.d. OPC Unified Architecture—Common Object Model: ISA-95, version 1.00, Section 4.2: ISA-95 Summary. https://reference.opcfoundation.org/specs/OPC-10030/4.2 (accessed 6 September 2026).

[35] NAURA, n.d. Cygnus Optima Prime PECVD System. https://www.naura.com/product/details_15_2442.html (accessed 11 September 2026).

[36] Syskey Technology Co., Ltd., n.d. PECVD. https://www.syskey.com.tw/?action=product&title=PECVD (accessed 11 September 2026).

[37] Achterberg, T., 2009. SCIP: Solving constraint integer programs. Mathematical Programming Computation 1, 1–41. doi:10.1007/s12532-008-0001-1.

[38] Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., Gottwald, L., Graczyk, C., Halbig, K., Hoen, A., Hojny, C., van der Hulst, R., Koch, T., Lübbecke, M.E., Maher, S.J., Matter, F., Mühmer, E., Müller, B., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Sofranac, B., Turner, M., Vigerske, S., Wegscheider, F., Wellner, P., Weninger, D., Witzig, J., 2023. Enabling research through the SCIP optimization suite 8.0. ACM Transactions on Mathematical Software 49, 22:1–22:21. doi:10.1145/3585516.

[39] Wang, J., Wang, Z., Li, X., Kuang, Y., Shi, Z., Zhu, F., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2024. Learning to Cut via Hierarchical Sequence/Set Model for Efficient Mixed-Integer Programming. arXiv:2404.12638. URL: https://arxiv.org/abs/2404.12638.

[40] Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks, in: Advances in Neural Information Processing Systems, pp. 2692–2700.

[41] Choo, J., Kwon, Y.D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y., 2022. Simulation-guided beam search for neural combinatorial optimization, in: Advances in Neural Information Processing Systems. URL: https://papers.nips.cc/paper_files/paper/2022/hash/39b9b60f0d149eabd1fff2d7c7d5afc4-Abstract-Conference.html, doi:10.52202/068431-0637.

[42] Nemhauser, G.L., Wolsey, L.A., Fisher, M.L., 1978. An analysis of approximations for maximizing submodular set functions—i. Mathematical Programming 14, 265–294. doi:10.1007/BF01588971.

[43] Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256. doi:10.1007/BF00992696.

[44] Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: Proceedings of the 33rd International Conference on Machine Learning, PMLR. pp. 1928–1937. URL: https://proceedings.mlr.press/v48/mniha16.html.